

LeetCode 310

Placement Roadmap - Complete Java Solutions

All 310 problems, in the recommended study order. Each solution is written in Java with the idea, key steps and edge cases explained in comments, followed by time and space complexity.

Study cycle: Understand pattern - Attempt yourself - Code in Java - Test edge cases - Analyze complexity - Study optimal - Write 2-3 line notes - Re-solve after 3-7 days.

How to read this book

- **Repeated problems.** Your list intentionally repeats some problems in different patterns (for example Two Sum appears as #1 and #51). The repeat entries say *Same as #N* and point to the full solution. Re-solve them from scratch - that is the point of the repeat.
- **Pattern entries.** #234 (Dijkstra basics) and #245 (Bellman-Ford pattern) are not single problems, so they are written as reusable templates.
- **Judge-provided helpers.** #103 and #104 use `guess()` and `isBadVersion()`, which LeetCode supplies. #197 (Boundary of Binary Tree) and #260 (Rat in a Maze) are not on the free LeetCode tier; they are written to the standard problem statement (GeeksforGeeks for Rat in a Maze).
- **Imports.** Assume `import java.util.*;` at the top of every solution.
- **Choice of solution.** Where a problem has several approaches I chose the one most commonly expected in interviews (usually the optimal one). Always state the brute force first in an interview, then improve it.
- **Verify.** Run each solution on LeetCode before relying on it; submitting is the real test, and I could not run them against the judge here.

Helper classes used by the solutions (already provided on LeetCode)

```
class ListNode { int val; ListNode next; ListNode(int v) { val = v; } ListNode(int v, ListNode n) { val = v; next = n; } }
class TreeNode { int val; TreeNode left, right; TreeNode(int v) { val = v; } }
// #136 Copy List with Random Pointer
class Node { int val; Node next, random; Node(int v) { val = v; } }
// #189 Populating Next Right Pointers
class Node { int val; Node left, right, next; }
// #218 Clone Graph
class Node { int val; List<Node> neighbors = new ArrayList<>(); Node(int v) { val = v; } }
```

Contents

1. Arrays	11
#1 Two Sum	11
#2 Best Time to Buy and Sell Stock	11
#3 Contains Duplicate	11
#4 Maximum Subarray	11
#5 Product of Array Except Self	12
#6 Maximum Product Subarray	12
#7 Find Minimum in Rotated Sorted Array	12
#8 Search in Rotated Sorted Array	12
#9 Maximum Sum Circular Subarray	13
#10 Majority Element	13
#11 Majority Element II	13
#12 Missing Number	14
#13 Find All Numbers Disappeared in an Array	14
#14 Single Number	14
#15 Sort Colors	14
#16 Move Zeroes	15
#17 Rotate Array	15
#18 Merge Sorted Array	15
#19 Intersection of Two Arrays	15
#20 Intersection of Two Arrays II	16
#21 Find the Duplicate Number	16
#22 Set Matrix Zeroes	16
#23 Spiral Matrix	16
#24 Rotate Image	17
#25 Pascal's Triangle	17
#26 Merge Intervals	17
#27 Insert Interval	17
#28 Non-overlapping Intervals	18
#29 Subarray Sum Equals K	18
#30 Maximum Product Difference Between Two Pairs	18
2. Strings	19
#31 Valid Anagram	19
#32 Valid Palindrome	19
#33 Longest Common Prefix	19
#34 Reverse String	19
#35 Reverse Words in a String	20
#36 First Unique Character in a String	20
#37 Isomorphic Strings	20
#38 Ransom Note	20

#39 Longest Palindromic Substring	21
#40 Palindromic Substrings	21
#41 String to Integer (atoi)	21
#42 Roman to Integer	22
#43 Integer to Roman	22
#44 Longest Repeating Character Replacement	22
#45 Find the Index of the First Occurrence in a String	22
#46 Group Anagrams	23
#47 Decode String	23
#48 String Compression	23
#49 Multiply Strings	24
#50 Zigzag Conversion	24

3. Hashing 25

#51 Two Sum	25
#52 Contains Duplicate	25
#53 Contains Duplicate II	25
#54 Group Anagrams	25
#55 Top K Frequent Elements	25
#56 Valid Sudoku	26
#57 Longest Consecutive Sequence	26
#58 Subarray Sum Equals K	26
#59 Isomorphic Strings	26
#60 Word Pattern	26
#61 Happy Number	27
#62 Longest Substring Without Repeating Characters	27
#63 Four Sum II	27
#64 Subarray Sums Divisible by K	27
#65 Find All Anagrams in a String	28
#66 Minimum Window Substring	28
#67 Intersection of Two Arrays	28
#68 Intersection of Two Arrays II	28
#69 Find Common Characters	28
#70 First Missing Positive	29

4. Two Pointers 30

#71 Valid Palindrome	30
#72 Two Sum II - Input Array Is Sorted	30
#73 3Sum	30
#74 4Sum	30
#75 Container With Most Water	31
#76 Trapping Rain Water	31
#77 Remove Duplicates from Sorted Array	31
#78 Remove Duplicates from Sorted Array II	31
#79 Remove Element	32

#80 Move Zeroes	32
#81 Merge Sorted Array	32
#82 Squares of a Sorted Array	32
#83 Backspace String Compare	32
#84 Is Subsequence	33
#85 Boats to Save People	33

5. Sliding Window 34

#86 Maximum Average Subarray I	34
#87 Longest Substring Without Repeating Characters	34
#88 Longest Repeating Character Replacement	34
#89 Permutation in String	34
#90 Find All Anagrams in a String	34
#91 Minimum Window Substring	34
#92 Sliding Window Maximum	35
#93 Minimum Size Subarray Sum	35
#94 Fruit Into Baskets	35
#95 Max Consecutive Ones III	35
#96 Longest Subarray of 1's After Deleting One Element	36
#97 Subarray Product Less Than K	36
#98 Number of Sub-arrays of Size K and Average >= Threshold	36
#99 Maximum Number of Vowels in a Substring of Given Length	36
#100 Permutation in String	36

6. Binary Search 37

#101 Binary Search	37
#102 Search Insert Position	37
#103 Guess Number Higher or Lower	37
#104 First Bad Version	37
#105 Sqrt(x)	38
#106 Valid Perfect Square	38
#107 Find Peak Element	38
#108 Search a 2D Matrix	38
#109 Search a 2D Matrix II	39
#110 Find Minimum in Rotated Sorted Array	39
#111 Search in Rotated Sorted Array	39
#112 Search in Rotated Sorted Array II	39
#113 Find First and Last Position of Element in Sorted Array	39
#114 Koko Eating Bananas	40
#115 Capacity To Ship Packages Within D Days	40
#116 Find K Closest Elements	40
#117 Median of Two Sorted Arrays	41
#118 Split Array Largest Sum	41
#119 Time Based Key-Value Store	41
#120 Successful Pairs of Spells and Potions	42

7. Linked List	43
#121 Reverse Linked List	43
#122 Merge Two Sorted Lists	43
#123 Linked List Cycle	43
#124 Linked List Cycle II	43
#125 Remove Linked List Elements	44
#126 Remove Nth Node From End of List	44
#127 Middle of the Linked List	44
#128 Palindrome Linked List	44
#129 Intersection of Two Linked Lists	44
#130 Add Two Numbers	45
#131 Reorder List	45
#132 Odd Even Linked List	45
#133 Swap Nodes in Pairs	46
#134 Reverse Linked List II	46
#135 Rotate List	46
#136 Copy List with Random Pointer	47
#137 Merge k Sorted Lists	47
#138 Sort List	47
#139 Partition List	48
#140 LRU Cache	48
8. Stack & Queue	49
#141 Valid Parentheses	49
#142 Min Stack	49
#143 Implement Queue using Stacks	49
#144 Implement Stack using Queues	49
#145 Evaluate Reverse Polish Notation	50
#146 Daily Temperatures	50
#147 Next Greater Element I	50
#148 Next Greater Element II	50
#149 Largest Rectangle in Histogram	51
#150 Car Fleet	51
#151 Asteroid Collision	51
#152 Decode String	51
#153 Simplify Path	52
#154 Basic Calculator II	52
#155 Basic Calculator	52
#156 Sliding Window Maximum	52
#157 Online Stock Span	53
#158 Remove K Digits	53
#159 Trapping Rain Water	53
#160 Design Circular Queue	53

9. Trees	54
#161 Maximum Depth of Binary Tree	54
#162 Same Tree	54
#163 Invert Binary Tree	54
#164 Symmetric Tree	54
#165 Binary Tree Preorder Traversal	54
#166 Binary Tree Inorder Traversal	55
#167 Binary Tree Postorder Traversal	55
#168 Binary Tree Level Order Traversal	55
#169 Binary Tree Zigzag Level Order Traversal	56
#170 Minimum Depth of Binary Tree	56
#171 Balanced Binary Tree	56
#172 Diameter of Binary Tree	56
#173 Path Sum	57
#174 Path Sum II	57
#175 Sum Root to Leaf Numbers	57
#176 Binary Tree Right Side View	57
#177 Count Good Nodes in Binary Tree	58
#178 Lowest Common Ancestor of a Binary Tree	58
#179 Lowest Common Ancestor of a Binary Search Tree	58
#180 Validate Binary Search Tree	58
#181 Search in a Binary Search Tree	58
#182 Insert into a Binary Search Tree	59
#183 Delete Node in a BST	59
#184 Kth Smallest Element in a BST	59
#185 Convert Sorted Array to Binary Search Tree	59
#186 Construct Binary Tree from Preorder and Inorder Traversal	60
#187 Construct Binary Tree from Inorder and Postorder Traversal	60
#188 Flatten Binary Tree to Linked List	60
#189 Populating Next Right Pointers in Each Node	61
#190 Binary Tree Maximum Path Sum	61
#191 Serialize and Deserialize Binary Tree	61
#192 Subtree of Another Tree	62
#193 House Robber III	62
#194 All Nodes Distance K in Binary Tree	62
#195 Binary Tree Cameras	63
#196 Vertical Order Traversal of a Binary Tree	63
#197 Boundary of Binary Tree	63
#198 Morris Traversal (Inorder, O(1) space)	64
#199 Maximum Width of Binary Tree	64
#200 Path Sum III	65
10. Heap / Priority Queue	66
#201 Kth Largest Element in an Array	66

#202 Kth Smallest Element in a Sorted Matrix	66
#203 Top K Frequent Elements	66
#204 Last Stone Weight	66
#205 K Closest Points to Origin	67
#206 Find Median from Data Stream	67
#207 Merge k Sorted Lists	67
#208 Task Scheduler	67
#209 Reorganize String	68
#210 Kth Largest Element in a Stream	68
#211 Sort Characters By Frequency	68
#212 Seat Reservation Manager	69
#213 Smallest Number in Infinite Set	69
#214 IPO	69
#215 Find K Pairs with Smallest Sums	69

11. Graphs 70

#216 Number of Islands	70
#217 Max Area of Island	70
#218 Clone Graph	70
#219 Flood Fill	71
#220 Number of Provinces	71
#221 Rotting Oranges	71
#222 Course Schedule	72
#223 Course Schedule II	72
#224 Pacific Atlantic Water Flow	72
#225 Surrounded Regions	73
#226 Graph Valid Tree	73
#227 Number of Connected Components in an Undirected Graph	73
#228 Word Ladder	74
#229 Word Search	74
#230 Open the Lock	74
#231 Network Delay Time	75
#232 Cheapest Flights Within K Stops	75
#233 Path With Minimum Effort	75
#234 Dijkstra basics - shortest path template	76
#235 Redundant Connection	76
#236 Accounts Merge	76
#237 Min Cost to Connect All Points	77
#238 Number of Operations to Make Network Connected	77
#239 Is Graph Bipartite?	78
#240 Possible Bipartition	78
#241 Reconstruct Itinerary	78
#242 Alien Dictionary	79
#243 Swim in Rising Water	79
#244 Critical Connections in a Network	80

#245 Cheapest Flights / Bellman-Ford pattern	80
--	----

12. Recursion & Backtracking 81

#246 Subsets	81
#247 Subsets II	81
#248 Permutations	81
#249 Permutations II	82
#250 Combination Sum	82
#251 Combination Sum II	82
#252 Combination Sum III	83
#253 Letter Combinations of a Phone Number	83
#254 Generate Parentheses	83
#255 Palindrome Partitioning	84
#256 Word Search	84
#257 N-Queens	84
#258 N-Queens II	85
#259 Sudoku Solver	85
#260 Rat in a Maze	85
#261 Restore IP Addresses	86
#262 Letter Tile Possibilities	86
#263 Matchsticks to Square	86
#264 Partition to K Equal Sum Subsets	87
#265 Beautiful Arrangement	87

13. Greedy 88

#266 Best Time to Buy and Sell Stock II	88
#267 Jump Game	88
#268 Jump Game II	88
#269 Gas Station	88
#270 Assign Cookies	89
#271 Lemonade Change	89
#272 Partition Labels	89
#273 Merge Triplets to Form Target Triplet	89
#274 Non-overlapping Intervals	89
#275 Minimum Number of Arrows to Burst Balloons	90
#276 Hand of Straights	90
#277 Task Scheduler	90
#278 Candy	90
#279 Queue Reconstruction by Height	91
#280 Boats to Save People	91

14. Dynamic Programming 92

#281 Climbing Stairs	92
#282 Min Cost Climbing Stairs	92
#283 House Robber	92

#284 House Robber II	92
#285 Longest Palindromic Substring (DP version)	93
#286 Coin Change	93
#287 Coin Change II	93
#288 Partition Equal Subset Sum	93
#289 Target Sum	94
#290 Decode Ways	94
#291 Longest Increasing Subsequence	94
#292 Longest Common Subsequence	95
#293 Edit Distance	95
#294 Unique Paths	95
#295 Unique Paths II	95
#296 Minimum Path Sum	96
#297 Triangle	96
#298 Word Break	96
#299 Word Break II	96
#300 Combination Sum IV	97
#301 House Robber III	97
#302 Maximum Product Subarray	97
#303 Best Time to Buy and Sell Stock with Cooldown	97
#304 Best Time to Buy and Sell Stock with Transaction Fee	97
#305 Interleaving String	98
#306 Distinct Subsequences	98
#307 Longest Palindromic Subsequence	98
#308 Regular Expression Matching	99
#309 Burst Balloons	99
#310 Maximum Profit in Job Scheduling	99

1. Arrays

#1 Two Sum

EASY

Time O(n) | Space O(n)

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        // Idea: for each x, we need (target - x). Store seen values in a HashMap.
        Map<Integer, Integer> seen = new HashMap<>(); // value -> index
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (seen.containsKey(need)) return new int[]{seen.get(need), i};
            seen.put(nums[i], i); // put AFTER check so we never reuse same element
        }
        return new int[]{};
    }
}
```

#2 Best Time to Buy and Sell Stock

EASY

Time O(n) | Space O(1)

```
class Solution {
    public int maxProfit(int[] prices) {
        // Idea: track the cheapest price so far; profit today = price - minSoFar.
        int min = Integer.MAX_VALUE, best = 0;
        for (int p : prices) {
            min = Math.min(min, p); // best day to have bought
            best = Math.max(best, p - min); // best profit if we sell today
        }
        return best;
    }
}
```

#3 Contains Duplicate

EASY

Time O(n) | Space O(n)

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        // HashSet.add returns false if the element already exists.
        Set<Integer> set = new HashSet<>();
        for (int x : nums) if (!set.add(x)) return true;
        return false;
    }
}
```

#4 Maximum Subarray

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int maxSubArray(int[] nums) {
        // Kadane: at each i, either extend previous subarray or start fresh.
        int cur = nums[0], best = nums[0];
        for (int i = 1; i < nums.length; i++) {
            cur = Math.max(nums[i], cur + nums[i]); // drop negative prefix
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

#5 Product of Array Except Self

MEDIUM

Time $O(n)$ | Space $O(1)$ extra

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        // Pass 1: res[i] = product of everything to the LEFT of i.
        res[0] = 1;
        for (int i = 1; i < n; i++) res[i] = res[i - 1] * nums[i - 1];
        // Pass 2: multiply by product of everything to the RIGHT of i.
        int right = 1;
        for (int i = n - 1; i >= 0; i--) {
            res[i] *= right;
            right *= nums[i];
        }
        return res;
    }
}
```

#6 Maximum Product Subarray

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int maxProduct(int[] nums) {
        // A negative number can flip the min into the max, so track both.
        int max = nums[0], min = nums[0], ans = nums[0];
        for (int i = 1; i < nums.length; i++) {
            int x = nums[i];
            if (x < 0) { int t = max; max = min; min = t; } // swap on negative
            max = Math.max(x, max * x);
            min = Math.min(x, min * x);
            ans = Math.max(ans, max);
        }
        return ans;
    }
}
```

#7 Find Minimum in Rotated Sorted Array

MEDIUM

Time $O(\log n)$ | Space $O(1)$

```
class Solution {
    public int findMin(int[] nums) {
        int lo = 0, hi = nums.length - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            // If mid > hi, the minimum lies strictly to the right of mid.
            if (nums[mid] > nums[hi]) lo = mid + 1;
            else hi = mid; // mid could itself be the minimum
        }
        return nums[lo];
    }
}
```

#8 Search in Rotated Sorted Array

MEDIUM

Time $O(\log n)$ | Space $O(1)$

```
class Solution {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;
            if (nums[lo] <= nums[mid]) { // left half is sorted
                if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
                else lo = mid + 1;
            } else { // right half is sorted
                if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
                else hi = mid - 1;
            }
        }
        return -1;
    }
}
```

#9 Maximum Sum Circular Subarray

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        // Answer = max( normal Kadane max, total - minSubarray ).
        int total = 0, curMax = 0, curMin = 0;
        int best = nums[0], worst = nums[0];
        for (int x : nums) {
            total += x;
            curMax = Math.max(x, curMax + x); best = Math.max(best, curMax);
            curMin = Math.min(x, curMin + x); worst = Math.min(worst, curMin);
        }
        // If all numbers are negative, total == worst -> wrapping would be empty.
        return best < 0 ? best : Math.max(best, total - worst);
    }
}
```

#10 Majority Element

EASY

Time O(n) | Space O(1)

```
class Solution {
    public int majorityElement(int[] nums) {
        // Boyer-Moore voting: majority element survives pairwise cancellation.
        int cand = 0, count = 0;
        for (int x : nums) {
            if (count == 0) cand = x;
            count += (x == cand) ? 1 : -1;
        }
        return cand;
    }
}
```

#11 Majority Element II

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public List<Integer> majorityElement(int[] nums) {
        // At most 2 elements can appear > n/3 times -> Boyer-Moore with 2 candidates.
        int c1 = 0, c2 = 1, n1 = 0, n2 = 0;
        for (int x : nums) {
            if (x == c1) n1++;
            else if (x == c2) n2++;
            else if (n1 == 0) { c1 = x; n1 = 1; }
            else if (n2 == 0) { c2 = x; n2 = 1; }
            else { n1--; n2--; }
        }
        // Verify: candidates are only "possible" majorities.
        n1 = 0; n2 = 0;
        for (int x : nums) { if (x == c1) n1++; else if (x == c2) n2++; }
        List<Integer> res = new ArrayList<>();
        if (n1 > nums.length / 3) res.add(c1);
        if (n2 > nums.length / 3) res.add(c2);
        return res;
    }
}
```

#12 Missing Number

EASY

Time O(n) | Space O(1)

```
class Solution {
    public int missingNumber(int[] nums) {
        // XOR of 0..n with all elements leaves only the missing number.
        int x = nums.length;
        for (int i = 0; i < nums.length; i++) x ^= i ^ nums[i];
        return x;
    }
}
```

#13 Find All Numbers Disappeared in an Array

EASY

Time O(n) | Space O(1)

```
class Solution {
    public List<Integer> findDisappearedNumbers(int[] nums) {
        // Use the sign of nums[v-1] as a "seen v" flag.
        for (int x : nums) {
            int idx = Math.abs(x) - 1;
            if (nums[idx] > 0) nums[idx] = -nums[idx];
        }
        List<Integer> res = new ArrayList<>();
        for (int i = 0; i < nums.length; i++)
            if (nums[i] > 0) res.add(i + 1); // never flagged -> missing
        return res;
    }
}
```

#14 Single Number

EASY

Time O(n) | Space O(1)

```
class Solution {
    public int singleNumber(int[] nums) {
        // a ^ a = 0 and a ^ 0 = a, so pairs cancel out.
        int r = 0;
        for (int x : nums) r ^= x;
        return r;
    }
}
```

#15 Sort Colors

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public void sortColors(int[] nums) {
        // Dutch National Flag: [0..lo) = 0s, [lo..i) = 1s, (hi..end] = 2s.
        int lo = 0, i = 0, hi = nums.length - 1;
        while (i <= hi) {
            if (nums[i] == 0) swap(nums, i++, lo++);
            else if (nums[i] == 2) swap(nums, i, hi--); // don't advance i: unchecked value
            else i++;
        }
        private void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
    }
}
```

#16 Move Zeroes

EASY

Time O(n) | Space O(1)

```
class Solution {
    public void moveZeroes(int[] nums) {
        int w = 0; // next write position for a non-zero
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                int t = nums[w]; nums[w] = nums[i]; nums[i] = t;
                w++;
            }
        }
    }
}
```

#17 Rotate Array

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        k %= n;
        // Reverse all, then reverse first k, then reverse the rest.
        reverse(nums, 0, n - 1);
        reverse(nums, 0, k - 1);
        reverse(nums, k, n - 1);
    }
    private void reverse(int[] a, int l, int r) {
        while (l < r) { int t = a[l]; a[l++] = a[r]; a[r--] = t; }
    }
}
```

#18 Merge Sorted Array

EASY

Time O(m+n) | Space O(1)

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        // Fill nums1 from the BACK so we never overwrite unread values.
        int i = m - 1, j = n - 1, k = m + n - 1;
        while (j >= 0) {
            if (i >= 0 && nums1[i] > nums2[j]) nums1[k--] = nums1[i--];
            else nums1[k--] = nums2[j--];
        }
    }
}
```

#19 Intersection of Two Arrays

EASY

Time O(n+m) | Space O(n)

```
class Solution {
    public int[] intersection(int[] nums1, int[] nums2) {
        Set<Integer> a = new HashSet<>();
        for (int x : nums1) a.add(x);
        Set<Integer> res = new HashSet<>();
        for (int x : nums2) if (a.contains(x)) res.add(x);
        int[] out = new int[res.size()];
        int i = 0;
        for (int x : res) out[i++] = x;
        return out;
    }
}
```

#20 Intersection of Two Arrays II

EASY

Time $O(n+m)$ | Space $O(\min(n,m))$

```
class Solution {
    public int[] intersect(int[] nums1, int[] nums2) {
        // Count occurrences in nums1, consume counts while scanning nums2.
        Map<Integer, Integer> cnt = new HashMap<>();
        for (int x : nums1) cnt.merge(x, 1, Integer::sum);
        List<Integer> res = new ArrayList<>();
        for (int x : nums2) {
            int c = cnt.getOrDefault(x, 0);
            if (c > 0) { res.add(x); cnt.put(x, c - 1); }
        }
        return res.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

#21 Find the Duplicate Number

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int findDuplicate(int[] nums) {
        // Treat array as linked list i -> nums[i]; duplicate = cycle entrance.
        int slow = nums[0], fast = nums[0];
        do { slow = nums[slow]; fast = nums[nums[fast]]; } while (slow != fast);
        slow = nums[0]; // phase 2: find cycle start
        while (slow != fast) { slow = nums[slow]; fast = nums[fast]; }
        return slow;
    }
}
```

#22 Set Matrix Zeroes

MEDIUM

Time $O(m*n)$ | Space $O(1)$

```
class Solution {
    public void setZeroes(int[][] a) {
        int m = a.length, n = a[0].length;
        boolean row0 = false, col0 = false;
        for (int j = 0; j < n; j++) if (a[0][j] == 0) row0 = true;
        for (int i = 0; i < m; i++) if (a[i][0] == 0) col0 = true;
        // Use first row/col as markers for the rest of the matrix.
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                if (a[i][j] == 0) { a[i][0] = 0; a[0][j] = 0; }
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                if (a[i][0] == 0 || a[0][j] == 0) a[i][j] = 0;
        if (row0) for (int j = 0; j < n; j++) a[0][j] = 0;
        if (col0) for (int i = 0; i < m; i++) a[i][0] = 0;
    }
}
```

#23 Spiral Matrix

MEDIUM

Time $O(m*n)$ | Space $O(1)$ extra

```
class Solution {
    public List<Integer> spiralOrder(int[][] a) {
        List<Integer> res = new ArrayList<>();
        int top = 0, bot = a.length - 1, left = 0, right = a[0].length - 1;
        while (top <= bot && left <= right) {
            for (int j = left; j <= right; j++) res.add(a[top][j]);
            top++;
            for (int i = top; i <= bot; i++) res.add(a[i][right]);
            right--;
            if (top <= bot) { for (int j = right; j >= left; j--) res.add(a[bot][j]); bot--; }
            if (left <= right) { for (int i = bot; i >= top; i--) res.add(a[i][left]); left++; }
        }
        return res;
    }
}
```

#24 Rotate Image

MEDIUM

Time $O(n^2)$ | Space $O(1)$

```
class Solution {
    public void rotate(int[][] m) {
        int n = m.length;
        // Rotate 90 deg clockwise = transpose, then reverse every row.
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++) {
                int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t;
            }
        for (int[] row : m)
            for (int l = 0, r = n - 1; l < r; l++, r--) {
                int t = row[l]; row[l] = row[r]; row[r] = t;
            }
    }
}
```

#25 Pascal's Triangle

EASY

Time $O(n^2)$ | Space $O(n^2)$

```
class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> tri = new ArrayList<>();
        for (int i = 0; i < numRows; i++) {
            List<Integer> row = new ArrayList<>();
            for (int j = 0; j <= i; j++) {
                if (j == 0 || j == i) row.add(1);
                else row.add(tri.get(i - 1).get(j - 1) + tri.get(i - 1).get(j)); // sum of two above
            }
            tri.add(row);
        }
        return tri;
    }
}
```

#26 Merge Intervals

MEDIUM

Time $O(n \log n)$ | Space $O(n)$

```
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0])); // by start
        List<int[]> res = new ArrayList<>();
        int[] cur = intervals[0];
        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] <= cur[1]) cur[1] = Math.max(cur[1], intervals[i][1]); // overlap
            else { res.add(cur); cur = intervals[i]; }
        }
        res.add(cur);
        return res.toArray(new int[0][]);
    }
}
```

#27 Insert Interval

MEDIUM

Time $O(n)$ | Space $O(n)$

```
class Solution {
    public int[][] insert(int[][] intervals, int[] ni) {
        List<int[]> res = new ArrayList<>();
        int i = 0, n = intervals.length;
        while (i < n && intervals[i][1] < ni[0]) res.add(intervals[i++]); // strictly before
        while (i < n && intervals[i][0] <= ni[1]) { // overlapping: merge
            ni[0] = Math.min(ni[0], intervals[i][0]);
            ni[1] = Math.max(ni[1], intervals[i][1]);
            i++;
        }
        res.add(ni);
        while (i < n) res.add(intervals[i++]); // strictly after
        return res.toArray(new int[0][]);
    }
}
```

#28 Non-overlapping Intervals

MEDIUM

Time $O(n \log n)$ | Space $O(1)$

```
class Solution {
    public int eraseOverlapIntervals(int[][] iv) {
        // Greedy: keep the interval that ends earliest -> leaves most room.
        Arrays.sort(iv, (a, b) -> Integer.compare(a[1], b[1]));
        int end = iv[0][1], removed = 0;
        for (int i = 1; i < iv.length; i++) {
            if (iv[i][0] < end) removed++; // overlaps -> remove it
            else end = iv[i][1];
        }
        return removed;
    }
}
```

#29 Subarray Sum Equals K

MEDIUM

Time $O(n)$ | Space $O(n)$

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        // prefix[j] - prefix[i] == k => look up prefix[j] - k.
        Map<Integer, Integer> freq = new HashMap<>();
        freq.put(0, 1); // empty prefix
        int sum = 0, count = 0;
        for (int x : nums) {
            sum += x;
            count += freq.getOrDefault(sum - k, 0);
            freq.merge(sum, 1, Integer::sum);
        }
        return count;
    }
}
```

#30 Maximum Product Difference Between Two Pairs

EASY

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int maxProductDifference(int[] nums) {
        // Want (two largest product) - (two smallest product).
        int max1 = 0, max2 = 0, min1 = Integer.MAX_VALUE, min2 = Integer.MAX_VALUE;
        for (int x : nums) {
            if (x > max1) { max2 = max1; max1 = x; } else if (x > max2) max2 = x;
            if (x < min1) { min2 = min1; min1 = x; } else if (x < min2) min2 = x;
        }
        return max1 * max2 - min1 * min2;
    }
}
```

2. Strings

#31 Valid Anagram

EASY

Time O(n) | Space O(1)

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] cnt = new int[26];
        for (int i = 0; i < s.length(); i++) {
            cnt[s.charAt(i) - 'a']++; // add from s
            cnt[t.charAt(i) - 'a']--; // remove from t
        }
        for (int c : cnt) if (c != 0) return false;
        return true;
    }
}
```

#32 Valid Palindrome

EASY

Time O(n) | Space O(1)

```
class Solution {
    public boolean isPalindrome(String s) {
        int l = 0, r = s.length() - 1;
        while (l < r) {
            while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++; // skip junk
            while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
            if (Character.toLowerCase(s.charAt(l++)) != Character.toLowerCase(s.charAt(r--))) return false;
        }
        return true;
    }
}
```

#33 Longest Common Prefix

EASY

Time O(total chars) | Space O(1)

```
class Solution {
    public String longestCommonPrefix(String[] strs) {
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            // Shrink prefix until the current string starts with it.
            while (!strs[i].startsWith(prefix)) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
}
```

#34 Reverse String

EASY

Time O(n) | Space O(1)

```
class Solution {
    public void reverseString(char[] s) {
        for (int l = 0, r = s.length - 1; l < r; l++, r--) {
            char t = s[l]; s[l] = s[r]; s[r] = t;
        }
    }
}
```

#35 Reverse Words in a String

MEDIUM

Time $O(n)$ | Space $O(n)$

```
class Solution {
    public String reverseWords(String s) {
        // trim + split on 1-or-more spaces removes extra whitespace.
        String[] w = s.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (int i = w.length - 1; i >= 0; i--) {
            sb.append(w[i]);
            if (i > 0) sb.append(' ');
        }
        return sb.toString();
    }
}
```

#36 First Unique Character in a String

EASY

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int firstUniqChar(String s) {
        int[] cnt = new int[26];
        for (char c : s.toCharArray()) cnt[c - 'a']++;
        for (int i = 0; i < s.length(); i++)
            if (cnt[s.charAt(i) - 'a'] == 1) return i;
        return -1;
    }
}
```

#37 Isomorphic Strings

EASY

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public boolean isIsomorphic(String s, String t) {
        // Track last-seen position of each char; mappings must be consistent both ways.
        int[] ms = new int[256], mt = new int[256];
        for (int i = 0; i < s.length(); i++) {
            char a = s.charAt(i), b = t.charAt(i);
            if (ms[a] != mt[b]) return false;
            ms[a] = mt[b] = i + 1; // +1 so that 0 means "unseen"
        }
        return true;
    }
}
```

#38 Ransom Note

EASY

Time $O(n+m)$ | Space $O(1)$

```
class Solution {
    public boolean canConstruct(String ransomNote, String magazine) {
        int[] cnt = new int[26];
        for (char c : magazine.toCharArray()) cnt[c - 'a']++;
        for (char c : ransomNote.toCharArray())
            if (--cnt[c - 'a'] < 0) return false; // ran out of that letter
        return true;
    }
}
```

#39 Longest Palindromic Substring

MEDIUM

Time $O(n^2)$ | Space $O(1)$

```
class Solution {
    public String longestPalindrome(String s) {
        int start = 0, maxLen = 1;
        for (int c = 0; c < s.length(); c++) {
            // Expand around center: odd length (c,c) and even length (c,c+1).
            for (int k = 0; k < 2; k++) {
                int l = c, r = c + k;
                while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
                int len = r - l - 1;
                if (len > maxLen) { maxLen = len; start = l + 1; }
            }
        }
        return s.substring(start, start + maxLen);
    }
}
```

#40 Palindromic Substrings

MEDIUM

Time $O(n^2)$ | Space $O(1)$

```
class Solution {
    public int countSubstrings(String s) {
        int count = 0;
        for (int c = 0; c < s.length(); c++) {
            for (int k = 0; k < 2; k++) { // odd and even centers
                int l = c, r = c + k;
                while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
                    count++; // every successful expansion = 1 palindrome
                    l--; r++;
                }
            }
        }
        return count;
    }
}
```

#41 String to Integer (atoi)

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int myAtoi(String s) {
        int i = 0, n = s.length(), sign = 1;
        while (i < n && s.charAt(i) == ' ') i++; // 1. skip spaces
        if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) // 2. sign
            sign = s.charAt(i++) == '-' ? -1 : 1;
        int res = 0;
        while (i < n && Character.isDigit(s.charAt(i))) { // 3. digits
            int d = s.charAt(i++) - '0';
            // 4. overflow check BEFORE res*10 + d
            if (res > Integer.MAX_VALUE / 10 || (res == Integer.MAX_VALUE / 10 && d > 7))
                return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
            res = res * 10 + d;
        }
        return res * sign;
    }
}
```

#42 Roman to Integer

EASY

Time O(n) | Space O(1)

```
class Solution {
    public int romanToInt(String s) {
        Map<Character, Integer> v = Map.of('I',1,'V',5,'X',10,'L',50,'C',100,'D',500,'M',1000);
        int total = 0;
        for (int i = 0; i < s.length(); i++) {
            int cur = v.get(s.charAt(i));
            // Smaller value before a larger one means subtraction (IV = 4).
            if (i + 1 < s.length() && cur < v.get(s.charAt(i + 1))) total -= cur;
            else total += cur;
        }
        return total;
    }
}
```

#43 Integer to Roman

MEDIUM

Time O(1) | Space O(1)

```
class Solution {
    public String intToRoman(int num) {
        int[] val = {1000,900,500,400,100,90,50,40,10,9,5,4,1};
        String[] sym = {"M","CM","D","CD","C","XC","L","XL","X","IX","V","IV","I"};
        StringBuilder sb = new StringBuilder();
        // Greedy: take the largest symbol that fits, repeatedly.
        for (int i = 0; i < val.length; i++)
            while (num >= val[i]) { sb.append(sym[i]); num -= val[i]; }
        return sb.toString();
    }
}
```

#44 Longest Repeating Character Replacement

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int characterReplacement(String s, int k) {
        int[] cnt = new int[26];
        int l = 0, maxFreq = 0, best = 0;
        for (int r = 0; r < s.length(); r++) {
            maxFreq = Math.max(maxFreq, ++cnt[s.charAt(r) - 'A']);
            // Window valid if (size - most frequent char) <= k replacements.
            if (r - l + 1 - maxFreq > k) cnt[s.charAt(l++) - 'A']--;
            best = Math.max(best, r - l + 1);
        }
        return best;
    }
}
```

#45 Find the Index of the First Occurrence in a String

EASY

Time O(n+m) | Space O(m)

```
class Solution {
    public int strStr(String hay, String needle) {
        // KMP: lps[i] = length of longest proper prefix of needle that is also a suffix up to i.
        int m = needle.length();
        int[] lps = new int[m];
        for (int i = 1, len = 0; i < m; ) {
            if (needle.charAt(i) == needle.charAt(len)) lps[i++] = ++len;
            else if (len > 0) len = lps[len - 1];
            else lps[i++] = 0;
        }
        for (int i = 0, j = 0; i < hay.length(); ) {
            if (hay.charAt(i) == needle.charAt(j)) { i++; j++; if (j == m) return i - m; }
            else if (j > 0) j = lps[j - 1]; // fall back, don't move i
            else i++;
        }
        return -1;
    }
}
```

#46 Group Anagrams

MEDIUM

Time $O(n*k)$ | Space $O(n*k)$

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            int[] cnt = new int[26];
            for (char c : s.toCharArray()) cnt[c - 'a']++;
            String key = Arrays.toString(cnt); // anagrams share the same count signature
            map.computeIfAbsent(key, x -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

#47 Decode String

MEDIUM

Time $O(\text{output})$ | Space $O(n)$

```
class Solution {
    public String decodeString(String s) {
        Deque<Integer> counts = new ArrayDeque<>();
        Deque<StringBuilder> strs = new ArrayDeque<>();
        StringBuilder cur = new StringBuilder();
        int k = 0;
        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) k = k * 10 + (c - '0');
            else if (c == '[') { // save state, start a new level
                counts.push(k); strs.push(cur);
                cur = new StringBuilder(); k = 0;
            } else if (c == ']') { // repeat current, append to parent
                StringBuilder parent = strs.pop();
                int rep = counts.pop();
                for (int i = 0; i < rep; i++) parent.append(cur);
                cur = parent;
            } else cur.append(c);
        }
        return cur.toString();
    }
}
```

#48 String Compression

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int compress(char[] chars) {
        int w = 0, i = 0;
        while (i < chars.length) {
            char c = chars[i];
            int j = i;
            while (j < chars.length && chars[j] == c) j++; // run = [i, j)
            chars[w++] = c;
            if (j - i > 1)
                for (char d : String.valueOf(j - i).toCharArray()) chars[w++] = d;
            i = j;
        }
        return w;
    }
}
```

#49 Multiply Strings

MEDIUM

Time $O(m \cdot n)$ | Space $O(m+n)$

```
class Solution {
    public String multiply(String a, String b) {
        if (a.equals("0") || b.equals("0")) return "0";
        int[] res = new int[a.length() + b.length()];
        // digit i of a * digit j of b lands at positions i+j and i+j+1.
        for (int i = a.length() - 1; i >= 0; i--) {
            for (int j = b.length() - 1; j >= 0; j--) {
                int p = (a.charAt(i) - '0') * (b.charAt(j) - '0') + res[i + j + 1];
                res[i + j + 1] = p % 10;
                res[i + j] += p / 10;
            }
        }
        StringBuilder sb = new StringBuilder();
        for (int d : res) if (!(sb.length() == 0 && d == 0)) sb.append(d); // skip leading 0s
        return sb.toString();
    }
}
```

#50 Zigzag Conversion

MEDIUM

Time $O(n)$ | Space $O(n)$

```
class Solution {
    public String convert(String s, int numRows) {
        if (numRows == 1) return s;
        StringBuilder[] rows = new StringBuilder[numRows];
        for (int i = 0; i < numRows; i++) rows[i] = new StringBuilder();
        int r = 0, dir = 1;
        for (char c : s.toCharArray()) {
            rows[r].append(c);
            if (r == 0) dir = 1; else if (r == numRows - 1) dir = -1; // bounce at edges
            r += dir;
        }
        StringBuilder sb = new StringBuilder();
        for (StringBuilder row : rows) sb.append(row);
        return sb.toString();
    }
}
```

3. Hashing

#51 Two Sum

EASY

Same problem as #1 - **Two Sum**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#52 Contains Duplicate

EASY

Same problem as #3 - **Contains Duplicate**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#53 Contains Duplicate II

EASY

Time $O(n)$ | Space $O(k)$

```
class Solution {
    public boolean containsNearbyDuplicate(int[] nums, int k) {
        // Sliding HashSet holding at most the last k elements.
        Set<Integer> window = new HashSet<>();
        for (int i = 0; i < nums.length; i++) {
            if (!window.add(nums[i])) return true;
            if (window.size() > k) window.remove(nums[i - k]);
        }
        return false;
    }
}
```

#54 Group Anagrams

MEDIUM

Same problem as #46 - **Group Anagrams**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#55 Top K Frequent Elements

MEDIUM

Time $O(n)$ | Space $O(n)$

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) freq.merge(x, 1, Integer::sum);
        // Bucket sort: bucket[f] holds numbers appearing exactly f times.
        List<Integer>[] bucket = new List[nums.length + 1];
        for (var e : freq.entrySet()) {
            if (bucket[e.getValue()] == null) bucket[e.getValue()] = new ArrayList<>();
            bucket[e.getValue()].add(e.getKey());
        }
        int[] res = new int[k];
        int idx = 0;
        for (int f = bucket.length - 1; f >= 0 && idx < k; f--)
            if (bucket[f] != null)
                for (int x : bucket[f]) { if (idx < k) res[idx++] = x; }
        return res;
    }
}
```

#56 Valid Sudoku

MEDIUM

Time O(81) | Space O(81)

```
class Solution {
    public boolean isValidSudoku(char[][] board) {
        Set<String> seen = new HashSet<>();
        for (int i = 0; i < 9; i++)
            for (int j = 0; j < 9; j++) {
                char c = board[i][j];
                if (c == '.') continue;
                // Encode each constraint as a unique string; add() false => duplicate.
                if (!seen.add(c + " in row " + i) ||
                    !seen.add(c + " in col " + j) ||
                    !seen.add(c + " in box " + i / 3 + "-" + j / 3)) return false;
            }
        return true;
    }
}
```

#57 Longest Consecutive Sequence

MEDIUM

Time O(n) | Space O(n)

```
class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int x : nums) set.add(x);
        int best = 0;
        for (int x : set) {
            if (set.contains(x - 1)) continue; // only start counting at a sequence start
            int len = 1;
            while (set.contains(x + len)) len++;
            best = Math.max(best, len);
        }
        return best;
    }
}
```

#58 Subarray Sum Equals K

MEDIUM

Same problem as #29 - *Subarray Sum Equals K*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#59 Isomorphic Strings

EASY

Same problem as #37 - *Isomorphic Strings*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#60 Word Pattern

EASY

Time O(n) | Space O(n)

```
class Solution {
    public boolean wordPattern(String pattern, String s) {
        String[] w = s.split(" ");
        if (w.length != pattern.length()) return false;
        Map<Object, Integer> map = new HashMap<>();
        for (int i = 0; i < w.length; i++) {
            // put returns the previous index; both must have same "first seen" index.
            Integer a = map.put(pattern.charAt(i), i);
            Integer b = map.put(w[i], i);
            if (!Objects.equals(a, b)) return false;
        }
        return true;
    }
}
```

#61 Happy Number

EASY

Time $O(\log n)$ | Space $O(1)$

```
class Solution {
    public boolean isHappy(int n) {
        // Floyd cycle detection on the digit-square-sum sequence.
        int slow = n, fast = next(n);
        while (fast != 1 && slow != fast) {
            slow = next(slow);
            fast = next(next(fast));
        }
        return fast == 1;
    }
    private int next(int n) {
        int s = 0;
        while (n > 0) { int d = n % 10; s += d * d; n /= 10; }
        return s;
    }
}
```

#62 Longest Substring Without Repeating Characters

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int[] last = new int[128]; // last[c] = index+1 where c was last seen
        int l = 0, best = 0;
        for (int r = 0; r < s.length(); r++) {
            char c = s.charAt(r);
            l = Math.max(l, last[c]); // jump left edge past the previous duplicate
            best = Math.max(best, r - l + 1);
            last[c] = r + 1;
        }
        return best;
    }
}
```

#63 Four Sum II

MEDIUM

Time $O(n^2)$ | Space $O(n^2)$

```
class Solution {
    public int fourSumCount(int[] a, int[] b, int[] c, int[] d) {
        // Meet in the middle: store all a+b sums, then look up -(c+d).
        Map<Integer, Integer> map = new HashMap<>();
        for (int x : a) for (int y : b) map.merge(x + y, 1, Integer::sum);
        int count = 0;
        for (int x : c) for (int y : d) count += map.getOrDefault(-(x + y), 0);
        return count;
    }
}
```

#64 Subarray Sums Divisible by K

MEDIUM

Time $O(n)$ | Space $O(k)$

```
class Solution {
    public int subarraysDivByK(int[] nums, int k) {
        // Two prefixes with the same remainder mod k => subarray between is divisible.
        int[] rem = new int[k];
        rem[0] = 1;
        int sum = 0, count = 0;
        for (int x : nums) {
            sum = ((sum + x) % k + k) % k; // normalize negative remainders
            count += rem[sum]++;
        }
        return count;
    }
}
```

#65 Find All Anagrams in a String

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> res = new ArrayList<>();
        if (s.length() < p.length()) return res;
        int[] need = new int[26], win = new int[26];
        for (char c : p.toCharArray()) need[c - 'a']++;
        for (int i = 0; i < s.length(); i++) {
            win[s.charAt(i) - 'a']++;
            if (i >= p.length()) win[s.charAt(i - p.length()) - 'a']--; // slide window
            if (Arrays.equals(need, win)) res.add(i - p.length() + 1);
        }
        return res;
    }
}
```

#66 Minimum Window Substring

HARD

Time O(n+m) | Space O(1)

```
class Solution {
    public String minWindow(String s, String t) {
        int[] need = new int[128];
        for (char c : t.toCharArray()) need[c]++;
        int missing = t.length(), l = 0, bestL = 0, bestLen = Integer.MAX_VALUE;
        for (int r = 0; r < s.length(); r++) {
            if (need[s.charAt(r)]-- > 0) missing--; // this char was still needed
            while (missing == 0) { // window valid: shrink from left
                if (r - l + 1 < bestLen) { bestLen = r - l + 1; bestL = l; }
                if (++need[s.charAt(l++)] > 0) missing++; // we removed a needed char
            }
        }
        return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestL, bestL + bestLen);
    }
}
```

#67 Intersection of Two Arrays

EASY

Same problem as #19 - *Intersection of Two Arrays*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#68 Intersection of Two Arrays II

EASY

Same problem as #20 - *Intersection of Two Arrays II*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#69 Find Common Characters

EASY

Time O(total chars) | Space O(1)

```
class Solution {
    public List<String> commonChars(String[] words) {
        int[] min = new int[26];
        Arrays.fill(min, Integer.MAX_VALUE);
        for (String w : words) {
            int[] cnt = new int[26];
            for (char c : w.toCharArray()) cnt[c - 'a']++;
            for (int i = 0; i < 26; i++) min[i] = Math.min(min[i], cnt[i]); // min freq across words
        }
        List<String> res = new ArrayList<>();
        for (int i = 0; i < 26; i++)
            for (int k = 0; k < min[i]; k++) res.add(String.valueOf((char) ('a' + i)));
        return res;
    }
}
```

#70 First Missing Positive

HARD

Time O(n) | Space O(1)

```
class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        // Cyclic sort: put value v at index v-1 when 1 <= v <= n.
        for (int i = 0; i < n; i++) {
            while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i]) {
                int j = nums[i] - 1;
                int t = nums[j]; nums[j] = nums[i]; nums[i] = t;
            }
        }
        for (int i = 0; i < n; i++) if (nums[i] != i + 1) return i + 1;
        return n + 1;
    }
}
```

4. Two Pointers

#71 Valid Palindrome

EASY

Same problem as #32 - **Valid Palindrome**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#72 Two Sum II - Input Array Is Sorted

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int l = 0, r = numbers.length - 1;
        while (l < r) {
            int s = numbers[l] + numbers[r];
            if (s == target) return new int[]{l + 1, r + 1}; // 1-indexed
            if (s < target) l++; else r--; // sorted -> move the right pointer
        }
        return new int[]{};
    }
}
```

#73 3Sum

MEDIUM

Time $O(n^2)$ | Space $O(1)$

```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> res = new ArrayList<>();
        for (int i = 0; i < nums.length - 2; i++) {
            if (nums[i] > 0) break; // can't reach 0 anymore
            if (i > 0 && nums[i] == nums[i - 1]) continue; // skip duplicate anchors
            int l = i + 1, r = nums.length - 1;
            while (l < r) {
                int s = nums[i] + nums[l] + nums[r];
                if (s < 0) l++;
                else if (s > 0) r--;
                else {
                    res.add(Arrays.asList(nums[i], nums[l], nums[r]));
                    l++; r--;
                    while (l < r && nums[l] == nums[l - 1]) l++; // skip duplicates
                    while (l < r && nums[r] == nums[r + 1]) r--;
                }
            }
        }
        return res;
    }
}
```

#74 4Sum

MEDIUM

Time $O(n^3)$ | Space $O(1)$

```
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        Arrays.sort(nums);
        List<List<Integer>> res = new ArrayList<>();
        int n = nums.length;
        for (int i = 0; i < n - 3; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            for (int j = i + 1; j < n - 2; j++) {
                if (j > i + 1 && nums[j] == nums[j - 1]) continue;
                int l = j + 1, r = n - 1;
                while (l < r) {
                    long s = (long) nums[i] + nums[j] + nums[l] + nums[r]; // long avoids overflow
                    if (s < target) l++;
                    else if (s > target) r--;
                    else {
                        res.add(Arrays.asList(nums[i], nums[j], nums[l], nums[r]));
                        l++; r--;
                        while (l < r && nums[l] == nums[l - 1]) l++;
                        while (l < r && nums[r] == nums[r + 1]) r--;
                    }
                }
            }
        }
    }
}
```

```

    }
}
return res;
}
}

```

#75 Container With Most Water

MEDIUM

Time O(n) | Space O(1)

```

class Solution {
    public int maxArea(int[] h) {
        int l = 0, r = h.length - 1, best = 0;
        while (l < r) {
            best = Math.max(best, Math.min(h[l], h[r]) * (r - l));
            // Move the shorter wall: moving the taller one can never improve the area.
            if (h[l] < h[r]) l++; else r--;
        }
        return best;
    }
}

```

#76 Trapping Rain Water

HARD

Time O(n) | Space O(1)

```

class Solution {
    public int trap(int[] h) {
        int l = 0, r = h.length - 1, lMax = 0, rMax = 0, water = 0;
        while (l < r) {
            // Water at a cell is bounded by the SMALLER of the two side maxima.
            if (h[l] < h[r]) {
                lMax = Math.max(lMax, h[l]);
                water += lMax - h[l++];
            } else {
                rMax = Math.max(rMax, h[r]);
                water += rMax - h[r--];
            }
        }
        return water;
    }
}

```

#77 Remove Duplicates from Sorted Array

EASY

Time O(n) | Space O(1)

```

class Solution {
    public int removeDuplicates(int[] nums) {
        int w = 1; // nums[0] is always kept
        for (int i = 1; i < nums.length; i++)
            if (nums[i] != nums[i - 1]) nums[w++] = nums[i];
        return w;
    }
}

```

#78 Remove Duplicates from Sorted Array II

MEDIUM

Time O(n) | Space O(1)

```

class Solution {
    public int removeDuplicates(int[] nums) {
        int w = 0;
        for (int x : nums)
            // Allow x if fewer than 2 elements written, or it differs from the one 2 back.
            if (w < 2 || x != nums[w - 2]) nums[w++] = x;
        return w;
    }
}

```

#79 Remove Element

EASY

Time O(n) | Space O(1)

```
class Solution {
    public int removeElement(int[] nums, int val) {
        int w = 0;
        for (int x : nums) if (x != val) nums[w++] = x;
        return w;
    }
}
```

#80 Move Zeroes

EASY

Same problem as #16 - **Move Zeroes**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#81 Merge Sorted Array

EASY

Same problem as #18 - **Merge Sorted Array**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#82 Squares of a Sorted Array

EASY

Time O(n) | Space O(n)

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        int l = 0, r = n - 1;
        // Largest square is at one of the two ends; fill result from the back.
        for (int i = n - 1; i >= 0; i--) {
            if (Math.abs(nums[l]) > Math.abs(nums[r])) { res[i] = nums[l] * nums[l]; l++; }
            else { res[i] = nums[r] * nums[r]; r--; }
        }
        return res;
    }
}
```

#83 Backspace String Compare

EASY

Time O(n) | Space O(1)

```
class Solution {
    public boolean backspaceCompare(String s, String t) {
        int i = s.length() - 1, j = t.length() - 1;
        while (i >= 0 || j >= 0) {
            i = nextValid(s, i);
            j = nextValid(t, j);
            if (i < 0 || j < 0) return i < 0 && j < 0; // both must finish together
            if (s.charAt(i--) != t.charAt(j--)) return false;
        }
        return true;
    }
    // Walk backwards, skipping chars erased by '#'.
    private int nextValid(String s, int i) {
        int skip = 0;
        while (i >= 0) {
            if (s.charAt(i) == '#') { skip++; i--; }
            else if (skip > 0) { skip--; i--; }
            else break;
        }
        return i;
    }
}
```

#84 Is Subsequence

EASY

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public boolean isSubsequence(String s, String t) {
        int i = 0;
        for (int j = 0; j < t.length() && i < s.length(); j++)
            if (s.charAt(i) == t.charAt(j)) i++; // matched next char of s
        return i == s.length();
    }
}
```

#85 Boats to Save People

MEDIUM

Time $O(n \log n)$ | Space $O(1)$

```
class Solution {
    public int numRescueBoats(int[] people, int limit) {
        Arrays.sort(people);
        int l = 0, r = people.length - 1, boats = 0;
        while (l <= r) {
            if (people[l] + people[r] <= limit) l++; // lightest can share with heaviest
            r--; // heaviest always leaves
            boats++;
        }
        return boats;
    }
}
```

5. Sliding Window

#86 Maximum Average Subarray I

EASY

Time O(n) | Space O(1)

```
class Solution {
    public double findMaxAverage(int[] nums, int k) {
        long sum = 0;
        for (int i = 0; i < k; i++) sum += nums[i];
        long best = sum;
        for (int i = k; i < nums.length; i++) {
            sum += nums[i] - nums[i - k]; // slide: add new, drop old
            best = Math.max(best, sum);
        }
        return (double) best / k;
    }
}
```

#87 Longest Substring Without Repeating Characters

MEDIUM

Same problem as #62 - **Longest Substring Without Repeating Characters**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#88 Longest Repeating Character Replacement

MEDIUM

Same problem as #44 - **Longest Repeating Character Replacement**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#89 Permutation in String

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public boolean checkInclusion(String s1, String s2) {
        if (s1.length() > s2.length()) return false;
        int[] need = new int[26], win = new int[26];
        int k = s1.length();
        for (char c : s1.toCharArray()) need[c - 'a']++;
        for (int i = 0; i < s2.length(); i++) {
            win[s2.charAt(i) - 'a']++;
            if (i >= k) win[s2.charAt(i - k) - 'a']--; // keep window size == k
            if (Arrays.equals(need, win)) return true; // same letter counts = permutation
        }
        return false;
    }
}
```

#90 Find All Anagrams in a String

MEDIUM

Same problem as #65 - **Find All Anagrams in a String**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#91 Minimum Window Substring

HARD

Same problem as #66 - **Minimum Window Substring**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#92 Sliding Window Maximum

HARD

Time O(n) | Space O(k)

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        // Monotonic decreasing deque of INDICES; front is always the window max.
        Deque<Integer> dq = new ArrayDeque<>();
        int[] res = new int[nums.length - k + 1];
        for (int i = 0; i < nums.length; i++) {
            if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst(); // out of window
            while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast(); // useless smaller
            dq.offerLast(i);
            if (i >= k - 1) res[i - k + 1] = nums[dq.peekFirst()];
        }
        return res;
    }
}
```

#93 Minimum Size Subarray Sum

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int l = 0, sum = 0, best = Integer.MAX_VALUE;
        for (int r = 0; r < nums.length; r++) {
            sum += nums[r];
            while (sum >= target) { // shrink while still valid
                best = Math.min(best, r - l + 1);
                sum -= nums[l++];
            }
        }
        return best == Integer.MAX_VALUE ? 0 : best;
    }
}
```

#94 Fruit Into Baskets

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int totalFruit(int[] fruits) {
        // Longest subarray with at most 2 distinct values.
        Map<Integer, Integer> cnt = new HashMap<>();
        int l = 0, best = 0;
        for (int r = 0; r < fruits.length; r++) {
            cnt.merge(fruits[r], 1, Integer::sum);
            while (cnt.size() > 2) {
                int f = fruits[l++];
                if (cnt.merge(f, -1, Integer::sum) == 0) cnt.remove(f);
            }
            best = Math.max(best, r - l + 1);
        }
        return best;
    }
}
```

#95 Max Consecutive Ones III

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int longestOnes(int[] nums, int k) {
        // Longest window containing at most k zeros.
        int l = 0, zeros = 0, best = 0;
        for (int r = 0; r < nums.length; r++) {
            if (nums[r] == 0) zeros++;
            while (zeros > k) if (nums[l++] == 0) zeros--;
            best = Math.max(best, r - l + 1);
        }
        return best;
    }
}
```

#96 Longest Subarray of 1's After Deleting One Element

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int longestSubarray(int[] nums) {
        // Longest window with at most ONE zero; answer is size-1 (we must delete one element).
        int l = 0, zeros = 0, best = 0;
        for (int r = 0; r < nums.length; r++) {
            if (nums[r] == 0) zeros++;
            while (zeros > 1) if (nums[l++] == 0) zeros--;
            best = Math.max(best, r - l); // (r-l+1) - 1
        }
        return best;
    }
}
```

#97 Subarray Product Less Than K

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int numSubarrayProductLessThanK(int[] nums, int k) {
        if (k <= 1) return 0;
        int l = 0, count = 0, prod = 1;
        for (int r = 0; r < nums.length; r++) {
            prod *= nums[r];
            while (prod >= k) prod /= nums[l++];
            count += r - l + 1; // all subarrays ending at r that start in [l..r]
        }
        return count;
    }
}
```

#98 Number of Sub-arrays of Size K and Average >= Threshold

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int numOfSubarrays(int[] arr, int k, int threshold) {
        int target = k * threshold, sum = 0, count = 0; // compare sums, avoids division
        for (int i = 0; i < arr.length; i++) {
            sum += arr[i];
            if (i >= k) sum -= arr[i - k];
            if (i >= k - 1 && sum >= target) count++;
        }
        return count;
    }
}
```

#99 Maximum Number of Vowels in a Substring of Given Length

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int maxVowels(String s, int k) {
        int cur = 0, best = 0;
        for (int i = 0; i < s.length(); i++) {
            if (isVowel(s.charAt(i))) cur++;
            if (i >= k && isVowel(s.charAt(i - k))) cur--; // element leaving the window
            best = Math.max(best, cur);
        }
        return best;
    }
    private boolean isVowel(char c) { return "aeiou".indexOf(c) >= 0; }
}
```

#100 Permutation in String

MEDIUM

Same problem as #89 - **Permutation in String**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

6. Binary Search

#101 Binary Search

EASY

Time $O(\log n)$ | Space $O(1)$

```
class Solution {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2; // avoids int overflow
            if (nums[mid] == target) return mid;
            if (nums[mid] < target) lo = mid + 1; else hi = mid - 1;
        }
        return -1;
    }
}
```

#102 Search Insert Position

EASY

Time $O(\log n)$ | Space $O(1)$

```
class Solution {
    public int searchInsert(int[] nums, int target) {
        // Find the first index with nums[i] >= target (lower bound).
        int lo = 0, hi = nums.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] < target) lo = mid + 1; else hi = mid;
        }
        return lo;
    }
}
```

#103 Guess Number Higher or Lower

EASY

Time $O(\log n)$ | Space $O(1)$

```
public class Solution extends GuessGame {
    // guess(num): -1 => pick is lower, 1 => pick is higher, 0 => correct (provided by LeetCode)
    public int guessNumber(int n) {
        int lo = 1, hi = n;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            int g = guess(mid);
            if (g == 0) return mid;
            if (g < 0) hi = mid - 1; else lo = mid + 1;
        }
        return -1;
    }
}
```

#104 First Bad Version

EASY

Time $O(\log n)$ | Space $O(1)$

```
public class Solution extends VersionControl {
    // isBadVersion(v) is provided by LeetCode. Find the FIRST true.
    public int firstBadVersion(int n) {
        int lo = 1, hi = n;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (isBadVersion(mid)) hi = mid; // mid could be the first bad
            else lo = mid + 1;
        }
        return lo;
    }
}
```

#105 Sqrt(x)

EASY

Time $O(\log x)$ | Space $O(1)$

```
class Solution {
    public int mySqrt(int x) {
        long lo = 0, hi = x;
        while (lo < hi) {
            long mid = (lo + hi + 1) / 2; // upper mid to avoid infinite loop
            if (mid * mid <= x) lo = mid; else hi = mid - 1;
        }
        return (int) lo;
    }
}
```

#106 Valid Perfect Square

EASY

Time $O(\log n)$ | Space $O(1)$

```
class Solution {
    public boolean isPerfectSquare(int num) {
        long lo = 1, hi = num;
        while (lo <= hi) {
            long mid = (lo + hi) / 2, sq = mid * mid;
            if (sq == num) return true;
            if (sq < num) lo = mid + 1; else hi = mid - 1;
        }
        return false;
    }
}
```

#107 Find Peak Element

MEDIUM

Time $O(\log n)$ | Space $O(1)$

```
class Solution {
    public int findPeakElement(int[] nums) {
        int lo = 0, hi = nums.length - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            // Going uphill to the right => a peak exists on the right side.
            if (nums[mid] < nums[mid + 1]) lo = mid + 1; else hi = mid;
        }
        return lo;
    }
}
```

#108 Search a 2D Matrix

MEDIUM

Time $O(\log(m*n))$ | Space $O(1)$

```
class Solution {
    public boolean searchMatrix(int[][] mat, int target) {
        int m = mat.length, n = mat[0].length;
        int lo = 0, hi = m * n - 1; // treat matrix as a flattened sorted array
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            int v = mat[mid / n][mid % n];
            if (v == target) return true;
            if (v < target) lo = mid + 1; else hi = mid - 1;
        }
        return false;
    }
}
```

#109 Search a 2D Matrix II

MEDIUM

Time $O(m+n)$ | Space $O(1)$

```
class Solution {
    public boolean searchMatrix(int[][] mat, int target) {
        // Start at top-right: left is smaller, down is bigger -> eliminate a row/col each step.
        int r = 0, c = mat[0].length - 1;
        while (r < mat.length && c >= 0) {
            if (mat[r][c] == target) return true;
            if (mat[r][c] > target) c--; else r++;
        }
        return false;
    }
}
```

#110 Find Minimum in Rotated Sorted Array

MEDIUM

Same problem as #7 - *Find Minimum in Rotated Sorted Array*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#111 Search in Rotated Sorted Array

MEDIUM

Same problem as #8 - *Search in Rotated Sorted Array*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#112 Search in Rotated Sorted Array II

MEDIUM

Time $O(\log n)$ avg, $O(n)$ worst | Space $O(1)$

```
class Solution {
    public boolean search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return true;
            if (nums[lo] == nums[mid] && nums[mid] == nums[hi]) { lo++; hi--; } // can't decide: shrink
            else if (nums[lo] <= nums[mid]) { // left sorted
                if (nums[lo] <= target && target < nums[mid]) hi = mid - 1; else lo = mid + 1;
            } else { // right sorted
                if (nums[mid] < target && target <= nums[hi]) lo = mid + 1; else hi = mid - 1;
            }
        }
        return false;
    }
}
```

#113 Find First and Last Position of Element in Sorted Array

MEDIUM

Time $O(\log n)$ | Space $O(1)$

```
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int first = lowerBound(nums, target);
        if (first == nums.length || nums[first] != target) return new int[]{-1, -1};
        return new int[]{first, lowerBound(nums, target + 1) - 1}; // last = (first index of target+1) - 1
    }
    // First index i with nums[i] >= x.
    private int lowerBound(int[] a, int x) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] < x) lo = mid + 1; else hi = mid;
        }
        return lo;
    }
}
```

#114 Koko Eating Bananas

MEDIUM

Time $O(n \log m)$ | Space $O(1)$

```
class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        // Binary search on the ANSWER (speed). Feasibility is monotonic in speed.
        int lo = 1, hi = 0;
        for (int p : piles) hi = Math.max(hi, p);
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            long hours = 0;
            for (int p : piles) hours += (p + mid - 1) / mid; // ceil(p / mid)
            if (hours <= h) hi = mid; else lo = mid + 1;
        }
        return lo;
    }
}
```

#115 Capacity To Ship Packages Within D Days

MEDIUM

Time $O(n \log S)$ | Space $O(1)$

```
class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int lo = 0, hi = 0;
        for (int w : weights) { lo = Math.max(lo, w); hi += w; } // capacity in [maxWeight, sum]
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            int need = 1, load = 0;
            for (int w : weights) {
                if (load + w > mid) { need++; load = 0; }
                load += w;
            }
            if (need <= days) hi = mid; else lo = mid + 1;
        }
        return lo;
    }
}
```

#116 Find K Closest Elements

MEDIUM

Time $O(\log(n-k) + k)$ | Space $O(1)$

```
class Solution {
    public List<Integer> findClosestElements(int[] arr, int k, int x) {
        // Binary search the START of the k-length window.
        int lo = 0, hi = arr.length - k;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            // If x is closer to arr[mid+k] than arr[mid], window should move right.
            if (x - arr[mid] > arr[mid + k] - x) lo = mid + 1; else hi = mid;
        }
        List<Integer> res = new ArrayList<>();
        for (int i = lo; i < lo + k; i++) res.add(arr[i]);
        return res;
    }
}
```

#117 Median of Two Sorted Arrays

HARDTime $O(\log(\min(m,n)))$ | Space $O(1)$

```
class Solution {
    public double findMedianSortedArrays(int[] a, int[] b) {
        if (a.length > b.length) return findMedianSortedArrays(b, a); // binary search the smaller
        int m = a.length, n = b.length, half = (m + n + 1) / 2;
        int lo = 0, hi = m;
        while (lo <= hi) {
            int i = (lo + hi) / 2; // elements taken from a's left part
            int j = half - i; // elements taken from b's left part
            int aL = i == 0 ? Integer.MIN_VALUE : a[i - 1];
            int aR = i == m ? Integer.MAX_VALUE : a[i];
            int bL = j == 0 ? Integer.MIN_VALUE : b[j - 1];
            int bR = j == n ? Integer.MAX_VALUE : b[j];
            if (aL <= bR && bL <= aR) { // correct partition found
                if ((m + n) % 2 == 1) return Math.max(aL, bL);
                return (Math.max(aL, bL) + Math.min(aR, bR)) / 2.0;
            }
            if (aL > bR) hi = i - 1; else lo = i + 1;
        }
        return 0;
    }
}
```

#118 Split Array Largest Sum

HARDTime $O(n \log S)$ | Space $O(1)$

```
class Solution {
    public int splitArray(int[] nums, int k) {
        int lo = 0, hi = 0;
        for (int x : nums) { lo = Math.max(lo, x); hi += x; }
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            // Greedy: how many parts needed if each part sum <= mid?
            int parts = 1, sum = 0;
            for (int x : nums) {
                if (sum + x > mid) { parts++; sum = 0; }
                sum += x;
            }
            if (parts <= k) hi = mid; else lo = mid + 1;
        }
        return lo;
    }
}
```

#119 Time Based Key-Value Store

MEDIUMTime set $O(1)$, get $O(\log n)$ | Space $O(n)$

```
class TimeMap {
    private Map<String, List<Object[]>> map = new HashMap<>(); // key -> list of {timestamp, value}
    public TimeMap() {}
    public void set(String key, String value, int timestamp) {
        // Timestamps are strictly increasing per key => list stays sorted.
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(new Object[]{timestamp, value});
    }
    public String get(String key, int timestamp) {
        List<Object[]> list = map.get(key);
        if (list == null) return "";
        int lo = 0, hi = list.size() - 1;
        String ans = "";
        while (lo <= hi) { // find the last timestamp <= given
            int mid = lo + (hi - lo) / 2;
            if ((int) list.get(mid)[0] <= timestamp) { ans = (String) list.get(mid)[1]; lo = mid + 1; }
            else hi = mid - 1;
        }
        return ans;
    }
}
```

#120 Successful Pairs of Spells and Potions

MEDIUM

Time $O((n+m) \log m)$ | Space $O(1)$

```
class Solution {
    public int[] successfulPairs(int[] spells, int[] potions, long success) {
        Arrays.sort(potions);
        int m = potions.length;
        int[] res = new int[spells.length];
        for (int i = 0; i < spells.length; i++) {
            // First potion index where spell * potion >= success.
            int lo = 0, hi = m;
            while (lo < hi) {
                int mid = lo + (hi - lo) / 2;
                if ((long) spells[i] * potions[mid] >= success) hi = mid; else lo = mid + 1;
            }
            res[i] = m - lo;    // all potions from lo onward succeed
        }
        return res;
    }
}
```

7. Linked List

#121 Reverse Linked List

EASY

Time O(n) | Space O(1)

```
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null, cur = head;
        while (cur != null) {
            ListNode next = cur.next; // save
            cur.next = prev;          // flip pointer
            prev = cur;               // advance
            cur = next;
        }
        return prev;
    }
}
```

#122 Merge Two Sorted Lists

EASY

Time O(n+m) | Space O(1)

```
class Solution {
    public ListNode mergeTwoLists(ListNode a, ListNode b) {
        ListNode dummy = new ListNode(0), tail = dummy; // dummy avoids head special-casing
        while (a != null && b != null) {
            if (a.val <= b.val) { tail.next = a; a = a.next; }
            else { tail.next = b; b = b.next; }
            tail = tail.next;
        }
        tail.next = (a != null) ? a : b; // attach remainder
        return dummy.next;
    }
}
```

#123 Linked List Cycle

EASY

Time O(n) | Space O(1)

```
public class Solution {
    public boolean hasCycle(ListNode head) {
        // Floyd: fast moves 2x. If there's a cycle, fast eventually laps slow.
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

#124 Linked List Cycle II

MEDIUM

Time O(n) | Space O(1)

```
public class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                // Distance head->entry == distance meeting point->entry (going around).
                ListNode p = head;
                while (p != slow) { p = p.next; slow = slow.next; }
                return p;
            }
        }
        return null;
    }
}
```

#125 Remove Linked List Elements

EASY

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode dummy = new ListNode(0, head), cur = dummy;
        while (cur.next != null) {
            if (cur.next.val == val) cur.next = cur.next.next; // unlink
            else cur = cur.next;
        }
        return dummy.next;
    }
}
```

#126 Remove Nth Node From End of List

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0, head), fast = dummy, slow = dummy;
        for (int i = 0; i < n; i++) fast = fast.next; // gap of n between the pointers
        while (fast.next != null) { fast = fast.next; slow = slow.next; }
        slow.next = slow.next.next; // slow sits just before the target
        return dummy.next;
    }
}
```

#127 Middle of the Linked List

EASY

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
        return slow; // second middle for even length
    }
}
```

#128 Palindrome Linked List

EASY

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        // 1) find middle 2) reverse second half 3) compare halves
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
        ListNode prev = null;
        while (slow != null) { ListNode nx = slow.next; slow.next = prev; prev = slow; slow = nx; }
        for (ListNode a = head, b = prev; b != null; a = a.next, b = b.next)
            if (a.val != b.val) return false;
        return true;
    }
}
```

#129 Intersection of Two Linked Lists

EASY

Time $O(n+m)$ | Space $O(1)$

```
public class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        // Each pointer walks A+B / B+A: both travel the same total length,
        // so they meet at the intersection (or both hit null together).
        ListNode a = headA, b = headB;
        while (a != b) {
            a = (a == null) ? headB : a.next;
            b = (b == null) ? headA : b.next;
        }
        return a;
    }
}
```

#130 Add Two Numbers

MEDIUM

Time $O(\max(n,m))$ | Space $O(1)$ extra

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0), tail = dummy;
        int carry = 0;
        while (l1 != null || l2 != null || carry != 0) {
            int sum = carry;
            if (l1 != null) { sum += l1.val; l1 = l1.next; }
            if (l2 != null) { sum += l2.val; l2 = l2.next; }
            tail.next = new ListNode(sum % 10);
            tail = tail.next;
            carry = sum / 10;
        }
        return dummy.next;
    }
}
```

#131 Reorder List

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public void reorderList(ListNode head) {
        if (head == null || head.next == null) return;
        // 1) find middle
        ListNode slow = head, fast = head;
        while (fast.next != null && fast.next.next != null) { slow = slow.next; fast = fast.next.next; }
        // 2) reverse second half
        ListNode prev = null, cur = slow.next;
        slow.next = null;
        while (cur != null) { ListNode nx = cur.next; cur.next = prev; prev = cur; cur = nx; }
        // 3) interleave first half and reversed second half
        ListNode a = head, b = prev;
        while (b != null) {
            ListNode an = a.next, bn = b.next;
            a.next = b; b.next = an;
            a = an; b = bn;
        }
    }
}
```

#132 Odd Even Linked List

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public ListNode oddEvenList(ListNode head) {
        if (head == null) return null;
        ListNode odd = head, even = head.next, evenHead = even;
        while (even != null && even.next != null) {
            odd.next = even.next; odd = odd.next; // chain odd positions
            even.next = odd.next; even = even.next; // chain even positions
        }
        odd.next = evenHead; // odds followed by evens
        return head;
    }
}
```

#133 Swap Nodes in Pairs

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public ListNode swapPairs(ListNode head) {
        ListNode dummy = new ListNode(0, head), prev = dummy;
        while (prev.next != null && prev.next.next != null) {
            ListNode a = prev.next, b = a.next;
            a.next = b.next;    // a -> after b
            b.next = a;        // b -> a
            prev.next = b;     // previous -> b
            prev = a;         // a is now the tail of the swapped pair
        }
        return dummy.next;
    }
}
```

#134 Reverse Linked List II

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public ListNode reverseBetween(ListNode head, int left, int right) {
        ListNode dummy = new ListNode(0, head), prev = dummy;
        for (int i = 1; i < left; i++) prev = prev.next;    // node before the segment
        ListNode cur = prev.next;
        // Repeatedly move the node after cur to the front of the segment.
        for (int i = 0; i < right - left; i++) {
            ListNode nx = cur.next;
            cur.next = nx.next;
            nx.next = prev.next;
            prev.next = nx;
        }
        return dummy.next;
    }
}
```

#135 Rotate List

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public ListNode rotateRight(ListNode head, int k) {
        if (head == null || head.next == null || k == 0) return head;
        int len = 1;
        ListNode tail = head;
        while (tail.next != null) { tail = tail.next; len++; }
        k %= len;
        if (k == 0) return head;
        tail.next = head;    // make it circular
        ListNode newTail = head;
        for (int i = 0; i < len - k - 1; i++) newTail = newTail.next;
        ListNode newHead = newTail.next;
        newTail.next = null;    // cut the circle
        return newHead;
    }
}
```

#136 Copy List with Random Pointer

MEDIUM

Time $O(n)$ | Space $O(1)$ extra

```
class Solution {
    public Node copyRandomList(Node head) {
        if (head == null) return null;
        // 1) Interleave: A -> A' -> B -> B' ...
        for (Node c = head; c != null; c = c.next.next) {
            Node copy = new Node(c.val);
            copy.next = c.next;
            c.next = copy;
        }
        // 2) Set random pointers of copies.
        for (Node c = head; c != null; c = c.next.next)
            if (c.random != null) c.next.random = c.random.next;
        // 3) Separate the two lists.
        Node newHead = head.next;
        for (Node c = head; c != null; c = c.next) {
            Node copy = c.next;
            c.next = copy.next;
            if (copy.next != null) copy.next = copy.next.next;
        }
        return newHead;
    }
}
```

#137 Merge k Sorted Lists

HARD

Time $O(N \log k)$ | Space $O(k)$

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        // Min-heap always holds the smallest current node of each list.
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);
        for (ListNode l : lists) if (l != null) pq.offer(l);
        ListNode dummy = new ListNode(0), tail = dummy;
        while (!pq.isEmpty()) {
            ListNode n = pq.poll();
            tail.next = n; tail = n;
            if (n.next != null) pq.offer(n.next);
        }
        return dummy.next;
    }
}
```

#138 Sort List

MEDIUM

Time $O(n \log n)$ | Space $O(\log n)$

```
class Solution {
    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) return head;
        // Split in the middle (slow ends at end of first half).
        ListNode slow = head, fast = head.next;
        while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
        ListNode second = slow.next;
        slow.next = null;
        return merge(sortList(head), sortList(second)); // merge sort
    }
    private ListNode merge(ListNode a, ListNode b) {
        ListNode dummy = new ListNode(0), t = dummy;
        while (a != null && b != null) {
            if (a.val <= b.val) { t.next = a; a = a.next; } else { t.next = b; b = b.next; }
            t = t.next;
        }
        t.next = (a != null) ? a : b;
        return dummy.next;
    }
}
```

#139 Partition List

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public ListNode partition(ListNode head, int x) {
        ListNode smallD = new ListNode(0), bigD = new ListNode(0);
        ListNode s = smallD, b = bigD;
        for (ListNode cur = head; cur != null; cur = cur.next) {
            if (cur.val < x) { s.next = cur; s = s.next; } // keeps relative order
            else { b.next = cur; b = b.next; }
        }
        b.next = null; // important: terminate to avoid a cycle
        s.next = bigD.next; // join the two lists
        return smallD.next;
    }
}
```

#140 LRU Cache

MEDIUM

Time $O(1)$ per op | Space $O(\text{capacity})$

```
class LRUCache {
    // HashMap for O(1) lookup + doubly linked list for O(1) recency updates.
    private class Node { int key, val; Node prev, next; Node(int k, int v) { key = k; val = v; } }
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0), tail = new Node(0, 0); // sentinels
    private final int cap;
    public LRUCache(int capacity) {
        cap = capacity;
        head.next = tail; tail.prev = head;
    }
    public int get(int key) {
        Node n = map.get(key);
        if (n == null) return -1;
        remove(n); addFront(n); // mark as most recently used
        return n.val;
    }
    public void put(int key, int value) {
        Node n = map.get(key);
        if (n != null) { n.val = value; remove(n); addFront(n); return; }
        if (map.size() == cap) { // evict least recently used (just before tail)
            Node lru = tail.prev;
            remove(lru); map.remove(lru.key);
        }
        n = new Node(key, value);
        map.put(key, n); addFront(n);
    }
    private void remove(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    private void addFront(Node n) {
        n.next = head.next; n.prev = head;
        head.next.prev = n; head.next = n;
    }
}
```

8. Stack & Queue

#141 Valid Parentheses

EASY

Time O(n) | Space O(n)

```
class Solution {
    public boolean isValid(String s) {
        Deque<Character> st = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(') st.push('('); // push the EXPECTED closing bracket
            else if (c == '[') st.push('[');
            else if (c == '{') st.push('{');
            else if (st.isEmpty() || st.pop() != c) return false;
        }
        return st.isEmpty();
    }
}
```

#142 Min Stack

MEDIUM

Time O(1) per op | Space O(n)

```
class MinStack {
    private Deque<int[]> st = new ArrayDeque<>(); // each entry: {value, minSoFar}
    public MinStack() {}
    public void push(int val) {
        int min = st.isEmpty() ? val : Math.min(val, st.peek()[1]);
        st.push(new int[]{val, min});
    }
    public void pop() { st.pop(); }
    public int top() { return st.peek()[0]; }
    public int getMin() { return st.peek()[1]; }
}
```

#143 Implement Queue using Stacks

EASY

Time amortized O(1) | Space O(n)

```
class MyQueue {
    private Deque<Integer> in = new ArrayDeque<>(), out = new ArrayDeque<>();
    public MyQueue() {}
    public void push(int x) { in.push(x); }
    public int pop() { move(); return out.pop(); }
    public int peek() { move(); return out.peek(); }
    public boolean empty() { return in.isEmpty() && out.isEmpty(); }
    // Only refill 'out' when empty: each element is moved at most once.
    private void move() { if (out.isEmpty()) while (!in.isEmpty()) out.push(in.pop()); }
}
```

#144 Implement Stack using Queues

EASY

Time push O(n), pop O(1) | Space O(n)

```
class MyStack {
    private Queue<Integer> q = new LinkedList<>();
    public MyStack() {}
    public void push(int x) {
        q.offer(x);
        // Rotate so the newest element comes to the front.
        for (int i = 0; i < q.size() - 1; i++) q.offer(q.poll());
    }
    public int pop() { return q.poll(); }
    public int top() { return q.peek(); }
    public boolean empty() { return q.isEmpty(); }
}
```

#145 Evaluate Reverse Polish Notation

MEDIUM

Time O(n) | Space O(n)

```
class Solution {
    public int evalRPN(String[] tokens) {
        Deque<Integer> st = new ArrayDeque<>();
        for (String t : tokens) {
            switch (t) {
                case "+": st.push(st.pop() + st.pop()); break;
                case "*": st.push(st.pop() * st.pop()); break;
                case "-": { int b = st.pop(), a = st.pop(); st.push(a - b); break; } // order matters
                case "/": { int b = st.pop(), a = st.pop(); st.push(a / b); break; }
                default: st.push(Integer.parseInt(t));
            }
        }
        return st.pop();
    }
}
```

#146 Daily Temperatures

MEDIUM

Time O(n) | Space O(n)

```
class Solution {
    public int[] dailyTemperatures(int[] t) {
        int[] res = new int[t.length];
        Deque<Integer> st = new ArrayDeque<>(); // indices with decreasing temperatures
        for (int i = 0; i < t.length; i++) {
            while (!st.isEmpty() && t[st.peek()] < t[i]) {
                int j = st.pop();
                res[j] = i - j; // day i is the next warmer day for j
            }
            st.push(i);
        }
        return res;
    }
}
```

#147 Next Greater Element I

EASY

Time O(n+m) | Space O(n)

```
class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {
        Map<Integer, Integer> next = new HashMap<>();
        Deque<Integer> st = new ArrayDeque<>();
        for (int x : nums2) {
            while (!st.isEmpty() && st.peek() < x) next.put(st.pop(), x);
            st.push(x);
        }
        int[] res = new int[nums1.length];
        for (int i = 0; i < nums1.length; i++) res[i] = next.getOrDefault(nums1[i], -1);
        return res;
    }
}
```

#148 Next Greater Element II

MEDIUM

Time O(n) | Space O(n)

```
class Solution {
    public int[] nextGreaterElements(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        Arrays.fill(res, -1);
        Deque<Integer> st = new ArrayDeque<>();
        // Circular: walk the array twice using index % n.
        for (int i = 0; i < 2 * n; i++) {
            int x = nums[i % n];
            while (!st.isEmpty() && nums[st.peek()] < x) res[st.pop()] = x;
            if (i < n) st.push(i);
        }
        return res;
    }
}
```

#149 Largest Rectangle in Histogram

HARD

Time O(n) | Space O(n)

```
class Solution {
    public int largestRectangleArea(int[] h) {
        Deque<Integer> st = new ArrayDeque<>(); // indices with increasing heights
        int best = 0, n = h.length;
        for (int i = 0; i <= n; i++) {
            int cur = (i == n) ? 0 : h[i]; // sentinel 0 flushes the stack
            while (!st.isEmpty() && h[st.peek()] > cur) {
                int height = h[st.pop()];
                int left = st.isEmpty() ? -1 : st.peek();
                best = Math.max(best, height * (i - left - 1)); // width between smaller bars
            }
            st.push(i);
        }
        return best;
    }
}
```

#150 Car Fleet

MEDIUM

Time O(n log n) | Space O(n)

```
class Solution {
    public int carFleet(int target, int[] position, int[] speed) {
        int n = position.length;
        double[][] cars = new double[n][2]; // {position, timeToTarget}
        for (int i = 0; i < n; i++) {
            cars[i][0] = position[i];
            cars[i][1] = (double) (target - position[i]) / speed[i];
        }
        Arrays.sort(cars, (a, b) -> Double.compare(b[0], a[0])); // closest to target first
        int fleets = 0;
        double slowest = 0;
        for (double[] c : cars) {
            // takes longer => can't catch up => new fleet
            if (c[1] > slowest) { fleets++; slowest = c[1]; }
        }
        return fleets;
    }
}
```

#151 Asteroid Collision

MEDIUM

Time O(n) | Space O(n)

```
class Solution {
    public int[] asteroidCollision(int[] asteroids) {
        Deque<Integer> st = new ArrayDeque<>();
        for (int a : asteroids) {
            boolean alive = true;
            // Collision only when stack top moves right and current moves left.
            while (alive && a < 0 && !st.isEmpty() && st.peek() > 0) {
                if (st.peek() < -a) st.pop(); // top explodes, keep checking
                else if (st.peek() == -a) { st.pop(); alive = false; } // both explode
                else alive = false; // current explodes
            }
            if (alive) st.push(a);
        }
        int[] res = new int[st.size()];
        for (int i = res.length - 1; i >= 0; i--) res[i] = st.pop();
        return res;
    }
}
```

#152 Decode String

MEDIUM

Same problem as #47 - **Decode String**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#153 Simplify Path

MEDIUM

Time O(n) | Space O(n)

```
class Solution {
    public String simplifyPath(String path) {
        Deque<String> st = new ArrayDeque<>();
        for (String p : path.split("/")) {
            if (p.isEmpty() || p.equals(".")) continue; // ignore
            if (p.equals("..")) { if (!st.isEmpty()) st.pop(); } // go up
            else st.push(p);
        }
        if (st.isEmpty()) return "/";
        StringBuilder sb = new StringBuilder();
        while (!st.isEmpty()) sb.append('/').append(st.pollLast()); // bottom -> top
        return sb.toString();
    }
}
```

#154 Basic Calculator II

MEDIUM

Time O(n) | Space O(n)

```
class Solution {
    public int calculate(String s) {
        Deque<Integer> st = new ArrayDeque<>();
        int num = 0; char op = '+'; // op applies to the number that just finished
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (Character.isDigit(c)) num = num * 10 + (c - '0');
            if ((!Character.isDigit(c) && c != ' ') || i == s.length() - 1) {
                if (op == '+') st.push(num);
                else if (op == '-') st.push(-num);
                else if (op == '*') st.push(st.pop() * num); // * and / bind tighter: apply now
                else st.push(st.pop() / num);
                op = c; num = 0;
            }
        }
        int sum = 0;
        for (int x : st) sum += x;
        return sum;
    }
}
```

#155 Basic Calculator

HARD

Time O(n) | Space O(n)

```
class Solution {
    public int calculate(String s) {
        Deque<Integer> st = new ArrayDeque<>(); // saves (result-so-far, sign) at each '('
        int res = 0, num = 0, sign = 1;
        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) num = num * 10 + (c - '0');
            else if (c == '+' || c == '-') { res += sign * num; num = 0; sign = (c == '+') ? 1 : -1; }
            else if (c == '(') { st.push(res); st.push(sign); res = 0; sign = 1; }
            else if (c == ')') {
                res += sign * num; num = 0;
                res *= st.pop(); // sign before '('
                res += st.pop(); // result before '('
            }
        }
        return res + sign * num;
    }
}
```

#156 Sliding Window Maximum

HARD

Same problem as #92 - *Sliding Window Maximum*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#157 Online Stock Span

MEDIUM

Time amortized $O(1)$ | Space $O(n)$

```
class StockSpanner {
    private Deque<int[]> st = new ArrayDeque<>(); // {price, span}
    public StockSpanner() {}
    public int next(int price) {
        int span = 1;
        // Absorb all previous days with price <= today's; their spans are merged.
        while (!st.isEmpty() && st.peek()[0] <= price) span += st.pop()[1];
        st.push(new int[]{price, span});
        return span;
    }
}
```

#158 Remove K Digits

MEDIUM

Time $O(n)$ | Space $O(n)$

```
class Solution {
    public String removeKdigits(String num, int k) {
        StringBuilder st = new StringBuilder(); // used as a monotonic increasing stack
        for (char c : num.toCharArray()) {
            // Removing a bigger digit before a smaller one always shrinks the number.
            while (k > 0 && st.length() > 0 && st.charAt(st.length() - 1) > c) {
                st.deleteCharAt(st.length() - 1); k--;
            }
            st.append(c);
        }
        while (k-- > 0 && st.length() > 0) st.deleteCharAt(st.length() - 1); // leftover removals
        int i = 0;
        while (i < st.length() && st.charAt(i) == '0') i++; // strip leading zeros
        String res = st.substring(i);
        return res.isEmpty() ? "0" : res;
    }
}
```

#159 Trapping Rain Water

HARD

Same problem as #76 - *Trapping Rain Water*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#160 Design Circular Queue

MEDIUM

Time $O(1)$ per op | Space $O(k)$

```
class MyCircularQueue {
    private final int[] data;
    private int head = 0, size = 0;
    public MyCircularQueue(int k) { data = new int[k]; }
    public boolean enqueue(int value) {
        if (isFull()) return false;
        data[(head + size) % data.length] = value; // tail position
        size++;
        return true;
    }
    public boolean dequeue() {
        if (isEmpty()) return false;
        head = (head + 1) % data.length;
        size--;
        return true;
    }
    public int Front() { return isEmpty() ? -1 : data[head]; }
    public int Rear() { return isEmpty() ? -1 : data[(head + size - 1) % data.length]; }
    public boolean isEmpty() { return size == 0; }
    public boolean isFull() { return size == data.length; }
}
```

9. Trees

#161 Maximum Depth of Binary Tree

EASY

Time O(n) | Space O(h)

```
class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right)); // height = 1 + taller child
    }
}
```

#162 Same Tree

EASY

Time O(n) | Space O(h)

```
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null) return p == q; // both null -> same, one null -> different
        return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

#163 Invert Binary Tree

EASY

Time O(n) | Space O(h)

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null;
        TreeNode left = invertTree(root.left), right = invertTree(root.right);
        root.left = right; // swap children after inverting subtrees
        root.right = left;
        return root;
    }
}
```

#164 Symmetric Tree

EASY

Time O(n) | Space O(h)

```
class Solution {
    public boolean isSymmetric(TreeNode root) { return root == null || mirror(root.left, root.right); }
    private boolean mirror(TreeNode a, TreeNode b) {
        if (a == null || b == null) return a == b;
        // outer pair and inner pair must both mirror each other
        return a.val == b.val && mirror(a.left, b.right) && mirror(a.right, b.left);
    }
}
```

#165 Binary Tree Preorder Traversal

EASY

Time O(n) | Space O(h)

```
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        Deque<TreeNode> st = new ArrayDeque<>();
        if (root != null) st.push(root);
        while (!st.isEmpty()) {
            TreeNode n = st.pop();
            res.add(n.val); // root
            if (n.right != null) st.push(n.right); // push right first so left is processed first
            if (n.left != null) st.push(n.left);
        }
        return res;
    }
}
```

#166 Binary Tree Inorder Traversal

EASY

Time O(n) | Space O(h)

```
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        Deque<TreeNode> st = new ArrayDeque<>();
        TreeNode cur = root;
        while (cur != null || !st.isEmpty()) {
            while (cur != null) { st.push(cur); cur = cur.left; } // go as far left as possible
            cur = st.pop();
            res.add(cur.val); // visit
            cur = cur.right; // then right subtree
        }
        return res;
    }
}
```

#167 Binary Tree Postorder Traversal

EASY

Time O(n) | Space O(h)

```
class Solution {
    public List<Integer> postorderTraversal(TreeNode root) {
        LinkedList<Integer> res = new LinkedList<>();
        Deque<TreeNode> st = new ArrayDeque<>();
        if (root != null) st.push(root);
        // Trick: reverse of (root, right, left) is (left, right, root) = postorder.
        while (!st.isEmpty()) {
            TreeNode n = st.pop();
            res.addFirst(n.val);
            if (n.left != null) st.push(n.left);
            if (n.right != null) st.push(n.right);
        }
        return res;
    }
}
```

#168 Binary Tree Level Order Traversal

MEDIUM

Time O(n) | Space O(w)

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);
        while (!q.isEmpty()) {
            int size = q.size(); // nodes in the current level
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode n = q.poll();
                level.add(n.val);
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
            res.add(level);
        }
        return res;
    }
}
```

#169 Binary Tree Zigzag Level Order Traversal

MEDIUM

Time O(n) | Space O(w)

```
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        Queue<TreeNode> q = new LinkedList<>();
        q.offer(root);
        boolean leftToRight = true;
        while (!q.isEmpty()) {
            int size = q.size();
            LinkedList<Integer> level = new LinkedList<>();
            for (int i = 0; i < size; i++) {
                TreeNode n = q.poll();
                if (leftToRight) level.addLast(n.val); else level.addFirst(n.val); // direction trick
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
            res.add(level);
            leftToRight = !leftToRight;
        }
        return res;
    }
}
```

#170 Minimum Depth of Binary Tree

EASY

Time O(n) | Space O(h)

```
class Solution {
    public int minDepth(TreeNode root) {
        if (root == null) return 0;
        // A node with one child is NOT a leaf: must go down the existing side.
        if (root.left == null) return 1 + minDepth(root.right);
        if (root.right == null) return 1 + minDepth(root.left);
        return 1 + Math.min(minDepth(root.left), minDepth(root.right));
    }
}
```

#171 Balanced Binary Tree

EASY

Time O(n) | Space O(h)

```
class Solution {
    public boolean isBalanced(TreeNode root) { return height(root) != -1; }
    // Returns height, or -1 as soon as any subtree is unbalanced (early exit).
    private int height(TreeNode n) {
        if (n == null) return 0;
        int l = height(n.left); if (l == -1) return -1;
        int r = height(n.right); if (r == -1) return -1;
        if (Math.abs(l - r) > 1) return -1;
        return 1 + Math.max(l, r);
    }
}
```

#172 Diameter of Binary Tree

EASY

Time O(n) | Space O(h)

```
class Solution {
    private int best = 0;
    public int diameterOfBinaryTree(TreeNode root) { depth(root); return best; }
    private int depth(TreeNode n) {
        if (n == null) return 0;
        int l = depth(n.left), r = depth(n.right);
        best = Math.max(best, l + r); // longest path THROUGH this node (in edges)
        return 1 + Math.max(l, r);
    }
}
```

#173 Path Sum

EASY

Time O(n) | Space O(h)

```
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
        if (root == null) return false;
        if (root.left == null && root.right == null) return root.val == targetSum; // leaf
        int rem = targetSum - root.val;
        return hasPathSum(root.left, rem) || hasPathSum(root.right, rem);
    }
}
```

#174 Path Sum II

MEDIUM

Time O(n^2) | Space O(h)

```
class Solution {
    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        List<List<Integer>> res = new ArrayList<>();
        dfs(root, targetSum, new ArrayList<>(), res);
        return res;
    }
    private void dfs(TreeNode n, int rem, List<Integer> path, List<List<Integer>> res) {
        if (n == null) return;
        path.add(n.val); // choose
        if (n.left == null && n.right == null && rem == n.val) res.add(new ArrayList<>(path));
        dfs(n.left, rem - n.val, path, res);
        dfs(n.right, rem - n.val, path, res);
        path.remove(path.size() - 1); // un-choose (backtrack)
    }
}
```

#175 Sum Root to Leaf Numbers

MEDIUM

Time O(n) | Space O(h)

```
class Solution {
    public int sumNumbers(TreeNode root) { return dfs(root, 0); }
    private int dfs(TreeNode n, int cur) {
        if (n == null) return 0;
        cur = cur * 10 + n.val; // build number along the path
        if (n.left == null && n.right == null) return cur;
        return dfs(n.left, cur) + dfs(n.right, cur);
    }
}
```

#176 Binary Tree Right Side View

MEDIUM

Time O(n) | Space O(h)

```
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        dfs(root, 0, res);
        return res;
    }
    // Visit right child first; the first node seen at each depth is visible from the right.
    private void dfs(TreeNode n, int depth, List<Integer> res) {
        if (n == null) return;
        if (depth == res.size()) res.add(n.val);
        dfs(n.right, depth + 1, res);
        dfs(n.left, depth + 1, res);
    }
}
```

#177 Count Good Nodes in Binary Tree

MEDIUM

Time O(n) | Space O(h)

```
class Solution {
    public int goodNodes(TreeNode root) { return dfs(root, Integer.MIN_VALUE); }
    private int dfs(TreeNode n, int maxSoFar) {
        if (n == null) return 0;
        int good = n.val >= maxSoFar ? 1 : 0; // no bigger value on path from root
        maxSoFar = Math.max(maxSoFar, n.val);
        return good + dfs(n.left, maxSoFar) + dfs(n.right, maxSoFar);
    }
}
```

#178 Lowest Common Ancestor of a Binary Tree

MEDIUM

Time O(n) | Space O(h)

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) return root;
        TreeNode l = lowestCommonAncestor(root.left, p, q);
        TreeNode r = lowestCommonAncestor(root.right, p, q);
        if (l != null && r != null) return root; // p and q are in different subtrees
        return l != null ? l : r; // both in one side (or found one)
    }
}
```

#179 Lowest Common Ancestor of a Binary Search Tree

MEDIUM

Time O(h) | Space O(1)

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        while (root != null) {
            if (p.val < root.val && q.val < root.val) root = root.left; // both left
            else if (p.val > root.val && q.val > root.val) root = root.right; // both right
            else return root; // split point
        }
        return null;
    }
}
```

#180 Validate Binary Search Tree

MEDIUM

Time O(n) | Space O(h)

```
class Solution {
    public boolean isValidBST(TreeNode root) { return check(root, Long.MIN_VALUE, Long.MAX_VALUE); }
    // Every node must lie strictly inside the (lo, hi) range inherited from its ancestors.
    private boolean check(TreeNode n, long lo, long hi) {
        if (n == null) return true;
        if (n.val <= lo || n.val >= hi) return false;
        return check(n.left, lo, n.val) && check(n.right, n.val, hi);
    }
}
```

#181 Search in a Binary Search Tree

EASY

Time O(h) | Space O(1)

```
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        while (root != null && root.val != val)
            root = val < root.val ? root.left : root.right;
        return root;
    }
}
```

#182 Insert into a Binary Search Tree

MEDIUM

Time O(h) | Space O(h)

```
class Solution {
    public TreeNode insertIntoBST(TreeNode root, int val) {
        if (root == null) return new TreeNode(val); // found the empty spot
        if (val < root.val) root.left = insertIntoBST(root.left, val);
        else root.right = insertIntoBST(root.right, val);
        return root;
    }
}
```

#183 Delete Node in a BST

MEDIUM

Time O(h) | Space O(h)

```
class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) return null;
        if (key < root.val) root.left = deleteNode(root.left, key);
        else if (key > root.val) root.right = deleteNode(root.right, key);
        else {
            if (root.left == null) return root.right; // 0 or 1 child: splice out
            if (root.right == null) return root.left;
            TreeNode succ = root.right; // 2 children: inorder successor
            while (succ.left != null) succ = succ.left;
            root.val = succ.val; // copy successor value up
            root.right = deleteNode(root.right, succ.val); // delete the successor node
        }
        return root;
    }
}
```

#184 Kth Smallest Element in a BST

MEDIUM

Time O(h + k) | Space O(h)

```
class Solution {
    public int kthSmallest(TreeNode root, int k) {
        Deque<TreeNode> st = new ArrayDeque<>();
        TreeNode cur = root;
        while (cur != null || !st.isEmpty()) {
            while (cur != null) { st.push(cur); cur = cur.left; }
            cur = st.pop();
            if (--k == 0) return cur.val; // inorder of a BST is sorted
            cur = cur.right;
        }
        return -1;
    }
}
```

#185 Convert Sorted Array to Binary Search Tree

EASY

Time O(n) | Space O(log n)

```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) { return build(nums, 0, nums.length - 1); }
    private TreeNode build(int[] a, int l, int r) {
        if (l > r) return null;
        int mid = l + (r - l) / 2; // middle as root keeps the tree height-balanced
        TreeNode n = new TreeNode(a[mid]);
        n.left = build(a, l, mid - 1);
        n.right = build(a, mid + 1, r);
        return n;
    }
}
```

#186 Construct Binary Tree from Preorder and Inorder Traversal

MEDIUM

Time O(n) | Space O(n)

```
class Solution {
    private int pre = 0;
    private Map<Integer, Integer> idx = new HashMap<>();
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        for (int i = 0; i < inorder.length; i++) idx.put(inorder[i], i);
        return build(preorder, 0, inorder.length - 1);
    }
    private TreeNode build(int[] preorder, int l, int r) {
        if (l > r) return null;
        TreeNode root = new TreeNode(preorder[pre++]); // next preorder value is the root
        int m = idx.get(root.val); // split inorder around it
        root.left = build(preorder, l, m - 1);
        root.right = build(preorder, m + 1, r);
        return root;
    }
}
```

#187 Construct Binary Tree from Inorder and Postorder Traversal

MEDIUM

Time O(n) | Space O(n)

```
class Solution {
    private int post;
    private Map<Integer, Integer> idx = new HashMap<>();
    public TreeNode buildTree(int[] inorder, int[] postorder) {
        for (int i = 0; i < inorder.length; i++) idx.put(inorder[i], i);
        post = postorder.length - 1;
        return build(postorder, 0, inorder.length - 1);
    }
    private TreeNode build(int[] postorder, int l, int r) {
        if (l > r) return null;
        TreeNode root = new TreeNode(postorder[post--]); // postorder read from the end: root first
        int m = idx.get(root.val);
        root.right = build(postorder, m + 1, r); // RIGHT first (reverse postorder)
        root.left = build(postorder, l, m - 1);
        return root;
    }
}
```

#188 Flatten Binary Tree to Linked List

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public void flatten(TreeNode root) {
        TreeNode cur = root;
        while (cur != null) {
            if (cur.left != null) {
                TreeNode pred = cur.left;
                while (pred.right != null) pred = pred.right;
                pred.right = cur.right;
                cur.right = cur.left;
                cur.left = null;
            }
            cur = cur.right;
        }
    }
}
```

#189 Populating Next Right Pointers in Each Node

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public Node connect(Node root) {
        Node levelStart = root;
        while (levelStart != null && levelStart.left != null) { // perfect binary tree
            for (Node n = levelStart; n != null; n = n.next) {
                n.left.next = n.right; // same parent
                if (n.next != null) n.right.next = n.next.left; // across parents
            }
            levelStart = levelStart.left;
        }
        return root;
    }
}
```

#190 Binary Tree Maximum Path Sum

HARD

Time O(n) | Space O(h)

```
class Solution {
    private int best = Integer.MIN_VALUE;
    public int maxPathSum(TreeNode root) { gain(root); return best; }
    // Max gain of a downward path starting at n (ignore negative branches).
    private int gain(TreeNode n) {
        if (n == null) return 0;
        int l = Math.max(0, gain(n.left)), r = Math.max(0, gain(n.right));
        best = Math.max(best, n.val + l + r); // path bending at n
        return n.val + Math.max(l, r); // can only extend one side upward
    }
}
```

#191 Serialize and Deserialize Binary Tree

HARD

Time O(n) | Space O(n)

```
public class Codec {
    // Preorder with "#" marking nulls uniquely identifies a tree.
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        write(root, sb);
        return sb.toString();
    }
    private void write(TreeNode n, StringBuilder sb) {
        if (n == null) { sb.append("#,"); return; }
        sb.append(n.val).append(',');
        write(n.left, sb);
        write(n.right, sb);
    }
    public TreeNode deserialize(String data) {
        Queue<String> q = new LinkedList<>(Arrays.asList(data.split(",")));
        return read(q);
    }
    private TreeNode read(Queue<String> q) {
        String s = q.poll();
        if (s.equals("#")) return null;
        TreeNode n = new TreeNode(Integer.parseInt(s));
        n.left = read(q);
        n.right = read(q);
        return n;
    }
}
```

#192 Subtree of Another Tree

EASY

Time $O(m*n)$ | Space $O(h)$

```
class Solution {
    public boolean isSubtree(TreeNode root, TreeNode sub) {
        if (root == null) return false;
        return same(root, sub) || isSubtree(root.left, sub) || isSubtree(root.right, sub);
    }
    private boolean same(TreeNode a, TreeNode b) {
        if (a == null || b == null) return a == b;
        return a.val == b.val && same(a.left, b.left) && same(a.right, b.right);
    }
}
```

#193 House Robber III

MEDIUM

Time $O(n)$ | Space $O(h)$

```
class Solution {
    public int rob(TreeNode root) {
        int[] r = dfs(root);
        return Math.max(r[0], r[1]);
    }
    // returns {best if this node is robbed, best if NOT robbed}
    private int[] dfs(TreeNode n) {
        if (n == null) return new int[]{0, 0};
        int[] l = dfs(n.left), r = dfs(n.right);
        int robbed = n.val + l[1] + r[1];
        int skipped = Math.max(l[0], l[1]) + Math.max(r[0], r[1]); // children must be skipped
        return new int[]{robbed, skipped}; // children free to choose
    }
}
```

#194 All Nodes Distance K in Binary Tree

MEDIUM

Time $O(n)$ | Space $O(n)$

```
class Solution {
    public List<Integer> distanceK(TreeNode root, TreeNode target, int k) {
        // Tree has no parent link -> build one, then BFS outward from target in 3 directions.
        Map<TreeNode, TreeNode> parent = new HashMap<>();
        Deque<TreeNode> st = new ArrayDeque<>();
        st.push(root);
        while (!st.isEmpty()) {
            TreeNode n = st.pop();
            if (n.left != null) { parent.put(n.left, n); st.push(n.left); }
            if (n.right != null) { parent.put(n.right, n); st.push(n.right); }
        }
        Queue<TreeNode> q = new LinkedList<>();
        Set<TreeNode> seen = new HashSet<>();
        q.offer(target); seen.add(target);
        int dist = 0;
        while (!q.isEmpty() && dist < k) {
            for (int i = q.size(); i > 0; i--) {
                TreeNode n = q.poll();
                for (TreeNode nb : new TreeNode[]{n.left, n.right, parent.get(n)})
                    if (nb != null && seen.add(nb)) q.offer(nb);
            }
            dist++;
        }
        List<Integer> res = new ArrayList<>();
        for (TreeNode n : q) res.add(n.val);
        return res;
    }
}
```

#195 Binary Tree Cameras

HARD

Time O(n) | Space O(h)

```
class Solution {
    private int cameras = 0;
    // States: 0 = not covered, 1 = has camera, 2 = covered (no camera)
    public int minCameraCover(TreeNode root) {
        if (dfs(root) == 0) cameras++; // root uncovered -> needs a camera
        return cameras;
    }
    private int dfs(TreeNode n) {
        if (n == null) return 2; // null counts as covered
        int l = dfs(n.left), r = dfs(n.right);
        if (l == 0 || r == 0) { cameras++; return 1; } // a child needs cover -> place camera here
        if (l == 1 || r == 1) return 2; // a child has a camera -> covered
        return 0; // both children covered, none has camera
    }
}
```

#196 Vertical Order Traversal of a Binary Tree

HARD

Time O(n log n) | Space O(n)

```
class Solution {
    public List<List<Integer>> verticalTraversal(TreeNode root) {
        List<int[]> nodes = new ArrayList<>(); // {col, row, val}
        Deque<Object[]> q = new ArrayDeque<>(); // BFS carrying (node, row, col)
        q.offer(new Object[]{root, 0, 0});
        while (!q.isEmpty()) {
            Object[] cur = q.poll();
            TreeNode n = (TreeNode) cur[0]; int r = (int) cur[1], c = (int) cur[2];
            nodes.add(new int[]{c, r, n.val});
            if (n.left != null) q.offer(new Object[]{n.left, r + 1, c - 1});
            if (n.right != null) q.offer(new Object[]{n.right, r + 1, c + 1});
        }
        // Sort by column, then row, then value (tie rule from the problem).
        nodes.sort((a, b) -> a[0] != b[0] ? a[0] - b[0] : a[1] != b[1] ? a[1] - b[1] : a[2] - b[2]);
        List<List<Integer>> res = new ArrayList<>();
        int prevCol = Integer.MIN_VALUE;
        for (int[] n : nodes) {
            if (n[0] != prevCol) { res.add(new ArrayList<>()); prevCol = n[0]; }
            res.get(res.size() - 1).add(n[2]);
        }
        return res;
    }
}
```

#197 Boundary of Binary Tree

MEDIUM

Time O(n) | Space O(h)

```
class Solution {
    public List<Integer> boundaryOfBinaryTree(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        if (root == null) return res;
        if (!isLeaf(root)) res.add(root.val);
        // 1) left boundary top-down (no leaves)
        TreeNode n = root.left;
        while (n != null) {
            if (!isLeaf(n)) res.add(n.val);
            n = n.left != null ? n.left : n.right;
        }
        // 2) all leaves left-to-right
        leaves(root, res);
        // 3) right boundary bottom-up (collect top-down, then reverse via stack)
        Deque<Integer> st = new ArrayDeque<>();
        n = root.right;
        while (n != null) {
            if (!isLeaf(n)) st.push(n.val);
            n = n.right != null ? n.right : n.left;
        }
        while (!st.isEmpty()) res.add(st.pop());
        return res;
    }
    private boolean isLeaf(TreeNode n) { return n.left == null && n.right == null; }
    private void leaves(TreeNode n, List<Integer> res) {
        if (n == null) return;
        if (isLeaf(n)) { res.add(n.val); return; }
        leaves(n.left, res);
    }
}
```

```

    leaves(n.right, res);
}
}

```

#198 Morris Traversal (Inorder, O(1) space)

MEDIUM

Time O(n) | Space O(1)

```

class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        TreeNode cur = root;
        while (cur != null) {
            if (cur.left == null) { // nothing on left: visit and go right
                res.add(cur.val);
                cur = cur.right;
            } else {
                TreeNode pred = cur.left; // inorder predecessor
                while (pred.right != null && pred.right != cur) pred = pred.right;
                if (pred.right == null) { // first time: create temporary thread back to cur
                    pred.right = cur;
                    cur = cur.left;
                } else { // thread exists: left subtree done
                    pred.right = null; // restore tree
                    res.add(cur.val);
                    cur = cur.right;
                }
            }
        }
        return res;
    }
}

```

#199 Maximum Width of Binary Tree

MEDIUM

Time O(n) | Space O(w)

```

class Solution {
    public int widthOfBinaryTree(TreeNode root) {
        Queue<Object[]> q = new LinkedList<>(); // {node, index in a complete-tree numbering}
        q.offer(new Object[]{root, 0L});
        long best = 0;
        while (!q.isEmpty()) {
            int size = q.size();
            long first = (long) q.peek()[1], last = first;
            for (int i = 0; i < size; i++) {
                Object[] cur = q.poll();
                TreeNode n = (TreeNode) cur[0];
                long idx = (long) cur[1] - first; // normalize to avoid overflow
                last = idx;
                if (n.left != null) q.offer(new Object[]{n.left, 2 * idx});
                if (n.right != null) q.offer(new Object[]{n.right, 2 * idx + 1});
            }
            best = Math.max(best, last + 1);
        }
        return (int) best;
    }
}

```

#200 Path Sum III

MEDIUM

Time O(n) | Space O(h)

```
class Solution {
    public int pathSum(TreeNode root, int targetSum) {
        Map<Long, Integer> prefix = new HashMap<>();
        prefix.put(0L, 1);
        return dfs(root, 0L, targetSum, prefix);
    }
    // Same prefix-sum trick as Subarray Sum Equals K, applied on root-to-node paths.
    private int dfs(TreeNode n, long sum, int target, Map<Long, Integer> prefix) {
        if (n == null) return 0;
        sum += n.val;
        int count = prefix.getOrDefault(sum - target, 0);
        prefix.merge(sum, 1, Integer::sum);
        count += dfs(n.left, sum, target, prefix) + dfs(n.right, sum, target, prefix);
        prefix.merge(sum, -1, Integer::sum); // backtrack when leaving this node
        return count;
    }
}
```

10. Heap / Priority Queue

#201 Kth Largest Element in an Array

MEDIUM

Time $O(n \log k)$ | Space $O(k)$

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        // Min-heap of size k: its root is always the k-th largest seen so far.
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int x : nums) {
            pq.offer(x);
            if (pq.size() > k) pq.poll(); // drop the smallest
        }
        return pq.peek();
    }
}
```

#202 Kth Smallest Element in a Sorted Matrix

MEDIUM

Time $O(k \log n)$ | Space $O(n)$

```
class Solution {
    public int kthSmallest(int[][] matrix, int k) {
        int n = matrix.length;
        // Min-heap of {value, row, col}; seed with first element of each row (k-way merge).
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
        for (int r = 0; r < Math.min(n, k); r++) pq.offer(new int[]{matrix[r][0], r, 0});
        int val = 0;
        while (k-- > 0) {
            int[] cur = pq.poll();
            val = cur[0];
            if (cur[2] + 1 < n) pq.offer(new int[]{matrix[cur[1]][cur[2] + 1], cur[1], cur[2] + 1});
        }
        return val;
    }
}
```

#203 Top K Frequent Elements

MEDIUM

Same problem as #55 - *Top K Frequent Elements*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#204 Last Stone Weight

EASY

Time $O(n \log n)$ | Space $O(n)$

```
class Solution {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
        for (int s : stones) pq.offer(s);
        while (pq.size() > 1) {
            int a = pq.poll(), b = pq.poll();
            if (a != b) pq.offer(a - b); // smash the two heaviest
        }
        return pq.isEmpty() ? 0 : pq.peek();
    }
}
```

#205 K Closest Points to Origin

MEDIUM

Time $O(n \log k)$ | Space $O(k)$

```
class Solution {
    public int[][] kClosest(int[][] points, int k) {
        // Max-heap by distance, size k: root is the farthest of the k kept points.
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> dist(b) - dist(a));
        for (int[] p : points) {
            pq.offer(p);
            if (pq.size() > k) pq.poll();
        }
        return pq.toArray(new int[0][]);
    }
    private int dist(int[] p) { return p[0] * p[0] + p[1] * p[1]; } // no sqrt needed for comparison
}
```

#206 Find Median from Data Stream

HARD

Time addNum $O(\log n)$, findMedian $O(1)$ | Space $O(n)$

```
class MedianFinder {
    // lo = max-heap of smaller half, hi = min-heap of larger half. Sizes differ by at most 1.
    private PriorityQueue<Integer> lo = new PriorityQueue<>(Collections.reverseOrder());
    private PriorityQueue<Integer> hi = new PriorityQueue<>();
    public MedianFinder() {}
    public void addNum(int num) {
        lo.offer(num);
        hi.offer(lo.poll()); // move the largest of lo to hi (keeps ordering)
        if (hi.size() > lo.size()) lo.offer(hi.poll()); // rebalance so lo has >= elements
    }
    public double findMedian() {
        if (lo.size() > hi.size()) return lo.peek();
        return (lo.peek() + hi.peek()) / 2.0;
    }
}
```

#207 Merge k Sorted Lists

HARD

Same problem as #137 - *Merge k Sorted Lists*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#208 Task Scheduler

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] cnt = new int[26];
        int maxFreq = 0;
        for (char t : tasks) maxFreq = Math.max(maxFreq, ++cnt[t - 'A']);
        int numMax = 0;
        for (int c : cnt) if (c == maxFreq) numMax++;
        // Frame the schedule: (maxFreq-1) blocks of size (n+1), then the last row of max tasks.
        int slots = (maxFreq - 1) * (n + 1) + numMax;
        return Math.max(slots, tasks.length); // if there is no idle time, answer = number of tasks
    }
}
```

#209 Reorganize String

MEDIUM

Time $O(n \log 26)$ | Space $O(1)$

```
class Solution {
    public String reorganizeString(String s) {
        int[] cnt = new int[26];
        for (char c : s.toCharArray()) cnt[c - 'a']++;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[1] - a[1]); // {char, count}
        for (int i = 0; i < 26; i++) if (cnt[i] > 0) pq.offer(new int[]{i, cnt[i]});
        StringBuilder sb = new StringBuilder();
        int[] prev = null; // char used last step, held back one turn
        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            sb.append((char) ('a' + cur[0]));
            cur[1]--;
            if (prev != null && prev[1] > 0) pq.offer(prev); // now safe to reuse
            prev = cur;
        }
        return sb.length() == s.length() ? sb.toString() : ""; // stuck => impossible
    }
}
```

#210 Kth Largest Element in a Stream

EASY

Time $O(\log k)$ per add | Space $O(k)$

```
class KthLargest {
    private final PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap of size k
    private final int k;
    public KthLargest(int k, int[] nums) {
        this.k = k;
        for (int x : nums) add(x);
    }
    public int add(int val) {
        pq.offer(val);
        if (pq.size() > k) pq.poll();
        return pq.peek();
    }
}
```

#211 Sort Characters By Frequency

MEDIUM

Time $O(n)$ | Space $O(n)$

```
class Solution {
    public String frequencySort(String s) {
        int[] cnt = new int[128];
        for (char c : s.toCharArray()) cnt[c]++;
        // Bucket by frequency (max frequency <= n).
        List<Character>[] bucket = new List[s.length() + 1];
        for (int c = 0; c < 128; c++)
            if (cnt[c] > 0) {
                if (bucket[cnt[c]] == null) bucket[cnt[c]] = new ArrayList<>();
                bucket[cnt[c]].add((char) c);
            }
        StringBuilder sb = new StringBuilder();
        for (int f = bucket.length - 1; f > 0; f--)
            if (bucket[f] != null)
                for (char c : bucket[f]) for (int i = 0; i < f; i++) sb.append(c);
        return sb.toString();
    }
}
```

#212 Seat Reservation Manager

MEDIUM

Time $O(\log n)$ per op | Space $O(n)$

```
class SeatManager {
    private final PriorityQueue<Integer> free = new PriorityQueue<>(); // released seats
    private int next = 1; // smallest never-used seat
    public SeatManager(int n) {}
    public int reserve() {
        if (!free.isEmpty()) return free.poll(); // smallest released seat is always < next
        return next++;
    }
    public void unreserve(int seatNumber) { free.offer(seatNumber); }
}
```

#213 Smallest Number in Infinite Set

MEDIUM

Time $O(\log n)$ per op | Space $O(n)$

```
class SmallestInfiniteSet {
    private final TreeSet<Integer> added = new TreeSet<>(); // numbers added back
    private int cur = 1; // smallest number never popped
    public SmallestInfiniteSet() {}
    public int popSmallest() {
        if (added.isEmpty()) return added.pollFirst();
        return cur++;
    }
    public void addBack(int num) {
        if (num < cur) added.add(num); // only matters if it was already popped
    }
}
```

#214 IPO

HARD

Time $O(n \log n)$ | Space $O(n)$

```
class Solution {
    public int findMaximizedCapital(int k, int w, int[] profits, int[] capital) {
        int n = profits.length;
        Integer[] idx = new Integer[n];
        for (int i = 0; i < n; i++) idx[i] = i;
        Arrays.sort(idx, (a, b) -> capital[a] - capital[b]); // by capital needed
        PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder()); // profits
        int i = 0;
        while (k-- > 0) {
            while (i < n && capital[idx[i]] <= w) pq.offer(profits[idx[i++]]); // all affordable
            if (pq.isEmpty()) break;
            w += pq.poll(); // greedy: take the most profitable affordable project
        }
        return w;
    }
}
```

#215 Find K Pairs with Smallest Sums

MEDIUM

Time $O(k \log k)$ | Space $O(k)$

```
class Solution {
    public List<List<Integer>> kSmallestPairs(int[] a, int[] b, int k) {
        List<List<Integer>> res = new ArrayList<>();
        // Heap of {sum, i, j}. Start with (i, 0) for each i; expand j on pop.
        PriorityQueue<int[]> pq = new PriorityQueue<>((x, y) -> x[0] - y[0]);
        for (int i = 0; i < Math.min(a.length, k); i++) pq.offer(new int[]{a[i] + b[0], i, 0});
        while (k-- > 0 && !pq.isEmpty()) {
            int[] cur = pq.poll();
            res.add(Arrays.asList(a[cur[1]], b[cur[2]]));
            if (cur[2] + 1 < b.length) pq.offer(new int[]{a[cur[1]] + b[cur[2] + 1], cur[1], cur[2] + 1});
        }
        return res;
    }
}
```

11. Graphs

#216 Number of Islands

MEDIUM

Time $O(m*n)$ | Space $O(m*n)$

```
class Solution {
    public int numIslands(char[][] grid) {
        int count = 0;
        for (int i = 0; i < grid.length; i++)
            for (int j = 0; j < grid[0].length; j++)
                if (grid[i][j] == '1') { count++; sink(grid, i, j); } // new island found
        return count;
    }
    // DFS flood: mark visited land as '0' so we never count it again.
    private void sink(char[][] g, int i, int j) {
        if (i < 0 || j < 0 || i >= g.length || j >= g[0].length || g[i][j] != '1') return;
        g[i][j] = '0';
        sink(g, i + 1, j); sink(g, i - 1, j); sink(g, i, j + 1); sink(g, i, j - 1);
    }
}
```

#217 Max Area of Island

MEDIUM

Time $O(m*n)$ | Space $O(m*n)$

```
class Solution {
    public int maxAreaOfIsland(int[][] grid) {
        int best = 0;
        for (int i = 0; i < grid.length; i++)
            for (int j = 0; j < grid[0].length; j++)
                best = Math.max(best, dfs(grid, i, j));
        return best;
    }
    private int dfs(int[][] g, int i, int j) {
        if (i < 0 || j < 0 || i >= g.length || j >= g[0].length || g[i][j] != 1) return 0;
        g[i][j] = 0; // visited
        return 1 + dfs(g, i + 1, j) + dfs(g, i - 1, j) + dfs(g, i, j + 1) + dfs(g, i, j - 1);
    }
}
```

#218 Clone Graph

MEDIUM

Time $O(V+E)$ | Space $O(V)$

```
class Solution {
    private Map<Node, Node> clones = new HashMap<>(); // original -> clone (also acts as visited)
    public Node cloneGraph(Node node) {
        if (node == null) return null;
        if (clones.containsKey(node)) return clones.get(node);
        Node clone = new Node(node.val);
        clones.put(node, clone); // register BEFORE recursing to handle cycles
        for (Node nb : node.neighbors) clone.neighbors.add(cloneGraph(nb));
        return clone;
    }
}
```

#219 Flood Fill

EASY

Time $O(m*n)$ | Space $O(m*n)$

```
class Solution {
    public int[][] floodFill(int[][] image, int sr, int sc, int color) {
        int old = image[sr][sc];
        // if same color, nothing to do (avoids infinite loop)
        if (old != color) fill(image, sr, sc, old, color);
        return image;
    }
    private void fill(int[][] img, int r, int c, int old, int color) {
        if (r < 0 || c < 0 || r >= img.length || c >= img[0].length || img[r][c] != old) return;
        img[r][c] = color;
        fill(img, r + 1, c, old, color); fill(img, r - 1, c, old, color);
        fill(img, r, c + 1, old, color); fill(img, r, c - 1, old, color);
    }
}
```

#220 Number of Provinces

MEDIUM

Time $O(n^2)$ | Space $O(n)$

```
class Solution {
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length, count = 0;
        boolean[] seen = new boolean[n];
        for (int i = 0; i < n; i++)
            if (!seen[i]) { dfs(isConnected, i, seen); count++; } // each DFS = one province
        return count;
    }
    private void dfs(int[][] g, int u, boolean[] seen) {
        seen[u] = true;
        for (int v = 0; v < g.length; v++)
            if (g[u][v] == 1 && !seen[v]) dfs(g, v, seen);
    }
}
```

#221 Rotting Oranges

MEDIUM

Time $O(m*n)$ | Space $O(m*n)$

```
class Solution {
    public int orangesRotting(int[][] grid) {
        int m = grid.length, n = grid[0].length, fresh = 0;
        Queue<int[]> q = new LinkedList<>();
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == 2) q.offer(new int[]{i, j}); // multi-source BFS
                else if (grid[i][j] == 1) fresh++;
            }
        int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
        int minutes = 0;
        while (!q.isEmpty() && fresh > 0) {
            minutes++; // one BFS layer = one minute
            for (int s = q.size(); s > 0; s--) {
                int[] cur = q.poll();
                for (int[] d : dirs) {
                    int r = cur[0] + d[0], c = cur[1] + d[1];
                    if (r >= 0 && c >= 0 && r < m && c < n && grid[r][c] == 1) {
                        grid[r][c] = 2; fresh--; q.offer(new int[]{r, c});
                    }
                }
            }
        }
        return fresh == 0 ? minutes : -1;
    }
}
```

#222 Course Schedule

MEDIUM

Time $O(V+E)$ | Space $O(V+E)$

```
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        // Kahn's algorithm (BFS topological sort). Can finish iff no cycle.
        List<List<Integer>> adj = new ArrayList<>();
        int[] indeg = new int[numCourses];
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
        for (int[] p : prerequisites) { adj.get(p[1]).add(p[0]); indeg[p[0]]++; }
        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) if (indeg[i] == 0) q.offer(i);
        int done = 0;
        while (!q.isEmpty()) {
            int u = q.poll(); done++;
            for (int v : adj.get(u)) if (--indeg[v] == 0) q.offer(v);
        }
        return done == numCourses; // leftover nodes are stuck in a cycle
    }
}
```

#223 Course Schedule II

MEDIUM

Time $O(V+E)$ | Space $O(V+E)$

```
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adj = new ArrayList<>();
        int[] indeg = new int[numCourses];
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
        for (int[] p : prerequisites) { adj.get(p[1]).add(p[0]); indeg[p[0]]++; }
        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) if (indeg[i] == 0) q.offer(i);
        int[] order = new int[numCourses];
        int idx = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            order[idx++] = u; // the topological order itself
            for (int v : adj.get(u)) if (--indeg[v] == 0) q.offer(v);
        }
        return idx == numCourses ? order : new int[0];
    }
}
```

#224 Pacific Atlantic Water Flow

MEDIUM

Time $O(m*n)$ | Space $O(m*n)$

```
class Solution {
    public List<List<Integer>> pacificAtlantic(int[][] h) {
        int m = h.length, n = h[0].length;
        boolean[][] pac = new boolean[m][n], atl = new boolean[m][n];
        // Reverse thinking: flow UPHILL from each ocean's border cells.
        for (int i = 0; i < m; i++) { dfs(h, pac, i, 0); dfs(h, atl, i, n - 1); }
        for (int j = 0; j < n; j++) { dfs(h, pac, 0, j); dfs(h, atl, m - 1, j); }
        List<List<Integer>> res = new ArrayList<>();
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                if (pac[i][j] && atl[i][j]) res.add(Arrays.asList(i, j));
        return res;
    }
    private void dfs(int[][] h, boolean[][] vis, int i, int j) {
        vis[i][j] = true;
        int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
        for (int[] d : dirs) {
            int r = i + d[0], c = j + d[1];
            if (r >= 0 && c >= 0 && r < h.length && c < h[0].length && !vis[r][c] && h[r][c] >= h[i][j])
                dfs(h, vis, r, c);
        }
    }
}
```

#225 Surrounded Regions

MEDIUM

Time $O(m*n)$ | Space $O(m*n)$

```
class Solution {
    public void solve(char[][] board) {
        int m = board.length, n = board[0].length;
        // 1) Any 'O' connected to the border can't be captured: mark it '#'.
        for (int i = 0; i < m; i++) { mark(board, i, 0); mark(board, i, n - 1); }
        for (int j = 0; j < n; j++) { mark(board, 0, j); mark(board, m - 1, j); }
        // 2) Remaining 'O' are surrounded -> 'X'; restore '#' -> 'O'.
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                board[i][j] = board[i][j] == 'O' ? 'X' : board[i][j] == '#' ? 'O' : board[i][j];
    }
    private void mark(char[][] b, int i, int j) {
        if (i < 0 || j < 0 || i >= b.length || j >= b[0].length || b[i][j] != 'O') return;
        b[i][j] = '#';
        mark(b, i + 1, j); mark(b, i - 1, j); mark(b, i, j + 1); mark(b, i, j - 1);
    }
}
```

#226 Graph Valid Tree

MEDIUM

Time $O(V+E)$ | Space $O(V)$

```
class Solution {
    private int[] parent;
    public boolean validTree(int n, int[][] edges) {
        if (edges.length != n - 1) return false; // a tree has exactly n-1 edges
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        for (int[] e : edges) {
            int a = find(e[0]), b = find(e[1]);
            if (a == b) return false; // edge closes a cycle
            parent[a] = b;
        }
        return true; // n-1 edges & no cycle => connected tree
    }
    private int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; } // path halving
        return x;
    }
}
```

#227 Number of Connected Components in an Undirected Graph

MEDIUM

Time $O(V+E)$ | Space $O(V)$

```
class Solution {
    private int[] parent;
    public int countComponents(int n, int[][] edges) {
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        int comps = n;
        for (int[] e : edges) {
            int a = find(e[0]), b = find(e[1]);
            if (a != b) { parent[a] = b; comps--; } // each successful union merges two components
        }
        return comps;
    }
    private int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
}
```

#228 Word Ladder

HARDTime $O(N * L^2)$ | Space $O(N * L)$

```
class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> dict = new HashSet<>(wordList);
        if (!dict.contains(endWord)) return 0;
        Queue<String> q = new LinkedList<>();
        q.offer(beginWord);
        int steps = 1;
        while (!q.isEmpty()) {
            for (int s = q.size(); s > 0; s--) {
                char[] w = q.poll().toCharArray();
                for (int i = 0; i < w.length; i++) {
                    char orig = w[i];
                    for (char c = 'a'; c <= 'z'; c++) { // try every 1-letter change
                        w[i] = c;
                        String next = new String(w);
                        if (next.equals(endWord)) return steps + 1;
                        if (dict.remove(next)) q.offer(next); // remove = mark visited
                    }
                    w[i] = orig;
                }
            }
            steps++;
        }
        return 0;
    }
}
```

#229 Word Search

MEDIUMTime $O(m * n * 4^L)$ | Space $O(L)$

```
class Solution {
    public boolean exist(char[][] board, String word) {
        for (int i = 0; i < board.length; i++)
            for (int j = 0; j < board[0].length; j++)
                if (dfs(board, word, i, j, 0)) return true;
        return false;
    }
    private boolean dfs(char[][] b, String w, int i, int j, int k) {
        if (k == w.length()) return true;
        if (i < 0 || j < 0 || i >= b.length || j >= b[0].length || b[i][j] != w.charAt(k)) return false;
        char tmp = b[i][j];
        b[i][j] = '#'; // mark as used in this path
        boolean found = dfs(b, w, i + 1, j, k + 1) || dfs(b, w, i - 1, j, k + 1)
            || dfs(b, w, i, j + 1, k + 1) || dfs(b, w, i, j - 1, k + 1);
        b[i][j] = tmp; // backtrack
        return found;
    }
}
```

#230 Open the Lock

MEDIUMTime $O(10^4 * 8)$ | Space $O(10^4)$

```
class Solution {
    public int openLock(String[] deadends, String target) {
        Set<String> seen = new HashSet<>(Arrays.asList(deadends));
        if (seen.contains("0000")) return -1;
        Queue<String> q = new LinkedList<>();
        q.offer("0000"); seen.add("0000"); // deadends double as the visited set
        int moves = 0;
        while (!q.isEmpty()) {
            for (int s = q.size(); s > 0; s--) {
                String cur = q.poll();
                if (cur.equals(target)) return moves;
                for (int i = 0; i < 4; i++)
                    for (int d = -1; d <= 1; d += 2) { // turn wheel up or down
                        char[] c = cur.toCharArray();
                        c[i] = (char) ('0' + (c[i] - '0' + d + 10) % 10);
                        String next = new String(c);
                        if (seen.add(next)) q.offer(next);
                    }
            }
            moves++;
        }
        return -1;
    }
}
```

```
}
}
```

#231 Network Delay Time

MEDIUM

Time $O(E \log V)$ | Space $O(V+E)$

```
class Solution {
    public int networkDelayTime(int[][] times, int n, int k) {
        List<int[]>[] adj = new List[n + 1];
        for (int i = 1; i <= n; i++) adj[i] = new ArrayList<>();
        for (int[] t : times) adj[t[0]].add(new int[]{t[1], t[2]});
        int[] dist = new int[n + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // {node, dist}
        pq.offer(new int[]{k, 0});
        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            if (cur[1] > dist[cur[0]]) continue; // stale entry
            for (int[] e : adj[cur[0]]) {
                int nd = cur[1] + e[1];
                if (nd < dist[e[0]]) { dist[e[0]] = nd; pq.offer(new int[]{e[0], nd}); }
            }
        }
        int max = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == Integer.MAX_VALUE) return -1; // unreachable node
            max = Math.max(max, dist[i]);
        }
        return max;
    }
}
```

#232 Cheapest Flights Within K Stops

MEDIUM

Time $O(K \cdot E)$ | Space $O(V)$

```
class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        // Bellman-Ford limited to k+1 edges. Use a COPY each round so we only extend by ONE
        // edge per round.
        int INF = Integer.MAX_VALUE / 2;
        int[] dist = new int[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int i = 0; i <= k; i++) {
            int[] next = dist.clone();
            for (int[] f : flights)
                if (dist[f[0]] + f[2] < next[f[1]]) next[f[1]] = dist[f[0]] + f[2];
            dist = next;
        }
        return dist[dst] >= INF ? -1 : dist[dst];
    }
}
```

#233 Path With Minimum Effort

MEDIUM

Time $O(m \cdot n \log(m \cdot n))$ | Space $O(m \cdot n)$

```
class Solution {
    public int minimumEffortPath(int[][] h) {
        int m = h.length, n = h[0].length;
        int[][] effort = new int[m][n];
        for (int[] row : effort) Arrays.fill(row, Integer.MAX_VALUE);
        effort[0][0] = 0;
        // Dijkstra where path cost = MAX edge difference (not sum).
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]); // {effort, r, c}
        pq.offer(new int[]{0, 0, 0});
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            if (cur[1] == m - 1 && cur[2] == n - 1) return cur[0];
            if (cur[0] > effort[cur[1]][cur[2]]) continue;
            for (int[] d : dirs) {
                int r = cur[1] + d[0], c = cur[2] + d[1];
                if (r < 0 || c < 0 || r >= m || c >= n) continue;
                int e = Math.max(cur[0], Math.abs(h[r][c] - h[cur[1]][cur[2]]));
                if (e < effort[r][c]) { effort[r][c] = e; pq.offer(new int[]{e, r, c}); }
            }
        }
    }
}
```

```

    }
    }
    return 0;
}

```

#234 Dijkstra basics - shortest path template

MEDIUM

Time $O(E \log V)$ | Space $O(V+E)$

```

class Dijkstra {
    // Single-source shortest paths on a graph with NON-NEGATIVE weights.
    // adj[u] = list of {v, weight}. Returns dist[] from src (Integer.MAX_VALUE = unreachable).
    public static int[] shortestPaths(List<int[]>[] adj, int src) {
        int n = adj.length;
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // {node, distance}
        pq.offer(new int[]{src, 0});
        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            int u = cur[0];
            if (cur[1] > dist[u]) continue; // outdated entry: a shorter path was already found
            for (int[] e : adj[u]) { // relax every outgoing edge
                int v = e[0], nd = dist[u] + e[1];
                if (nd < dist[v]) { dist[v] = nd; pq.offer(new int[]{v, nd}); }
            }
        }
        return dist;
    }
}
// Why it works: the closest unfinalized node in the heap can never be improved later
// because all remaining edge weights are >= 0. Fails with negative edges (use Bellman-Ford).

```

#235 Redundant Connection

MEDIUM

Time $O(n * \alpha(n))$ | Space $O(n)$

```

class Solution {
    private int[] parent;
    public int[] findRedundantConnection(int[][] edges) {
        parent = new int[edges.length + 1];
        for (int i = 0; i < parent.length; i++) parent[i] = i;
        for (int[] e : edges) {
            int a = find(e[0]), b = find(e[1]);
            if (a == b) return e; // both already connected -> this edge makes the cycle
            parent[a] = b;
        }
        return new int[0];
    }
    private int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
}

```

#236 Accounts Merge

MEDIUM

Time $O(N \log N)$ | Space $O(N)$

```

class Solution {
    private Map<String, String> parent = new HashMap<>();
    public List<List<String>> accountsMerge(List<List<String>> accounts) {
        Map<String, String> owner = new HashMap<>();
        // Union all emails inside the same account.
        for (List<String> acc : accounts) {
            String first = acc.get(1);
            for (int i = 1; i < acc.size(); i++) {
                String e = acc.get(i);
                parent.putIfAbsent(e, e);
                owner.put(e, acc.get(0));
                parent.put(find(e), find(first));
            }
        }
        // Group emails by their root.
        Map<String, TreeSet<String>> groups = new HashMap<>();
        for (String e : parent.keySet())
            groups.computeIfAbsent(find(e), k -> new TreeSet<>()).add(e); // TreeSet keeps sorted order
    }
}

```

```

        List<List<String>> res = new ArrayList<>();
        for (var entry : groups.entrySet()) {
            List<String> row = new ArrayList<>();
            row.add(owner.get(entry.getKey()));
            row.addAll(entry.getValue());
            res.add(row);
        }
        return res;
    }
    private String find(String x) {
        while (!parent.get(x).equals(x)) {
            parent.put(x, parent.get(parent.get(x)));
            x = parent.get(x);
        }
        return x;
    }
}

```

#237 Min Cost to Connect All Points

MEDIUM

Time $O(n^2)$ | Space $O(n)$

```

class Solution {
    public int minCostConnectPoints(int[][] p) {
        // Prim's MST on a dense graph,  $O(n^2)$  without a heap.
        int n = p.length, total = 0;
        int[] minDist = new int[n];
        boolean[] inMst = new boolean[n];
        Arrays.fill(minDist, Integer.MAX_VALUE);
        minDist[0] = 0;
        for (int it = 0; it < n; it++) {
            int u = -1;
            for (int i = 0; i < n; i++) // cheapest node not yet in the MST
                if (!inMst[i] && (u == -1 || minDist[i] < minDist[u])) u = i;
            inMst[u] = true;
            total += minDist[u];
            for (int v = 0; v < n; v++)
                if (!inMst[v]) {
                    int d = Math.abs(p[u][0] - p[v][0]) + Math.abs(p[u][1] - p[v][1]); // Manhattan
                    if (d < minDist[v]) minDist[v] = d;
                }
        }
        return total;
    }
}

```

#238 Number of Operations to Make Network Connected

MEDIUM

Time $O(n + e)$ | Space $O(n)$

```

class Solution {
    private int[] parent;
    public int makeConnected(int n, int[][] connections) {
        if (connections.length < n - 1) return -1; // need at least n-1 cables to connect n computers
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        int comps = n;
        for (int[] c : connections) {
            int a = find(c[0]), b = find(c[1]);
            if (a != b) { parent[a] = b; comps--; }
        }
        // need (components - 1) cables; spare cables are guaranteed by the check above
        return comps - 1;
    }
    private int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
}

```

#239 Is Graph Bipartite?

MEDIUM

Time $O(V+E)$ | Space $O(V)$

```
class Solution {
    public boolean isBipartite(int[][] graph) {
        int n = graph.length;
        int[] color = new int[n];           // 0 = uncolored, 1 / -1 = two colors
        for (int s = 0; s < n; s++) {
            if (color[s] != 0) continue;
            Queue<Integer> q = new LinkedList<>();
            q.offer(s); color[s] = 1;
            while (!q.isEmpty()) {
                int u = q.poll();
                for (int v : graph[u]) {
                    if (color[v] == 0) { color[v] = -color[u]; q.offer(v); } // opposite color
                    else if (color[v] == color[u]) return false;           // odd cycle
                }
            }
        }
        return true;
    }
}
```

#240 Possible Bipartition

MEDIUM

Time $O(V+E)$ | Space $O(V+E)$

```
class Solution {
    public boolean possibleBipartition(int n, int[][] dislikes) {
        // Same as bipartite check: "dislike" edges must connect the two different groups.
        List<Integer>[] adj = new List[n + 1];
        for (int i = 1; i <= n; i++) adj[i] = new ArrayList<>();
        for (int[] d : dislikes) { adj[d[0]].add(d[1]); adj[d[1]].add(d[0]); }
        int[] color = new int[n + 1];
        for (int s = 1; s <= n; s++) {
            if (color[s] != 0) continue;
            Queue<Integer> q = new LinkedList<>();
            q.offer(s); color[s] = 1;
            while (!q.isEmpty()) {
                int u = q.poll();
                for (int v : adj[u]) {
                    if (color[v] == 0) { color[v] = -color[u]; q.offer(v); }
                    else if (color[v] == color[u]) return false;
                }
            }
        }
        return true;
    }
}
```

#241 Reconstruct Itinerary

HARD

Time $O(E \log E)$ | Space $O(E)$

```
class Solution {
    private Map<String, PriorityQueue<String>> adj = new HashMap<>();
    private LinkedList<String> route = new LinkedList<>();
    public List<String> findItinerary(List<List<String>> tickets) {
        // Hierholzer's algorithm for an Eulerian path; min-heap gives lexical order.
        for (List<String> t : tickets)
            adj.computeIfAbsent(t.get(0), k -> new PriorityQueue<>()).add(t.get(1));
        dfs("JFK");
        return route;
    }
    private void dfs(String u) {
        PriorityQueue<String> pq = adj.get(u);
        while (pq != null && !pq.isEmpty()) dfs(pq.poll());
        route.addFirst(u); // add on the way BACK; stuck (dead-end) airports end up last
    }
}
```

#242 Alien Dictionary

HARD

Time $O(C)$ | Space $O(1)$ (26 letters)

```
class Solution {
    public String alienOrder(String[] words) {
        Map<Character, Set<Character>> adj = new HashMap<>();
        int[] indeg = new int[26];
        for (String w : words) for (char c : w.toCharArray()) adj.putIfAbsent(c, new HashSet<>());
        // First differing character of adjacent words gives one ordering edge.
        for (int i = 0; i + 1 < words.length; i++) {
            String a = words[i], b = words[i + 1];
            if (a.length() > b.length() && a.startsWith(b)) return ""; // invalid: "abc" before "ab"
            for (int j = 0; j < Math.min(a.length(), b.length()); j++) {
                if (a.charAt(j) != b.charAt(j)) {
                    if (adj.get(a.charAt(j)).add(b.charAt(j))) indeg[b.charAt(j) - 'a']++;
                    break;
                }
            }
        }
        Queue<Character> q = new LinkedList<>();
        for (char c : adj.keySet()) if (indeg[c - 'a'] == 0) q.offer(c);
        StringBuilder sb = new StringBuilder();
        while (!q.isEmpty()) {
            char c = q.poll();
            sb.append(c);
            for (char nb : adj.get(c)) if (--indeg[nb - 'a'] == 0) q.offer(nb);
        }
        return sb.length() == adj.size() ? sb.toString() : ""; // cycle => no valid order
    }
}
```

#243 Swim in Rising Water

HARD

Time $O(n^2 \log n)$ | Space $O(n^2)$

```
class Solution {
    public int swimInWater(int[][] grid) {
        int n = grid.length;
        boolean[][] seen = new boolean[n][n];
        // Dijkstra-like: cost of a path = max cell value on it. Always expand the lowest boundary cell.
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]); // {time, r, c}
        pq.offer(new int[]{grid[0][0], 0, 0});
        seen[0][0] = true;
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            if (cur[1] == n - 1 && cur[2] == n - 1) return cur[0];
            for (int[] d : dirs) {
                int r = cur[1] + d[0], c = cur[2] + d[1];
                if (r >= 0 && c >= 0 && r < n && c < n && !seen[r][c]) {
                    seen[r][c] = true;
                    pq.offer(new int[]{Math.max(cur[0], grid[r][c]), r, c});
                }
            }
        }
        return -1;
    }
}
```

#244 Critical Connections in a Network

HARDTime $O(V+E)$ | Space $O(V+E)$

```
class Solution {
    private int timer = 0;
    public List<List<Integer>> criticalConnections(int n, List<List<Integer>> connections) {
        // Tarjan's bridge-finding algorithm.
        List<Integer>[] adj = new List[n];
        for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
        for (List<Integer> c : connections) { adj[c.get(0)].add(c.get(1)); adj[c.get(1)].add(c.get(0)); }
        int[] disc = new int[n], low = new int[n];
        Arrays.fill(disc, -1);
        List<List<Integer>> bridges = new ArrayList<>();
        dfs(0, -1, adj, disc, low, bridges);
        return bridges;
    }
    // disc = discovery time, low = earliest discovery time reachable from u's subtree using back edges.
    private void dfs(int u, int parent, List<Integer>[] adj, int[] disc, int[] low, List<List<Integer>>
        out) {
        disc[u] = low[u] = timer++;
        for (int v : adj[u]) {
            if (v == parent) continue;
            if (disc[v] == -1) {
                dfs(v, u, adj, disc, low, out);
                low[u] = Math.min(low[u], low[v]);
                // v's subtree can't reach above u => bridge
                if (low[v] > disc[u]) out.add(Arrays.asList(u, v));
            } else low[u] = Math.min(low[u], disc[v]); // back edge
        }
    }
}
```

#245 Cheapest Flights / Bellman-Ford pattern

MEDIUMTime $O(V^*E)$ | Space $O(V)$

```
class BellmanFord {
    // Single-source shortest paths that ALLOWS negative edges and detects negative cycles.
    // edges[i] = {u, v, w}. Returns dist[], or null if a negative cycle is reachable.
    public static int[] shortestPaths(int n, int[][] edges, int src) {
        long INF = Long.MAX_VALUE / 2;
        long[] dist = new long[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        // A shortest path has at most n-1 edges, so n-1 rounds of relaxing EVERY edge suffice.
        for (int i = 0; i < n - 1; i++) {
            boolean changed = false;
            for (int[] e : edges)
                if (dist[e[0]] != INF && dist[e[0]] + e[2] < dist[e[1]]) {
                    dist[e[1]] = dist[e[0]] + e[2];
                    changed = true;
                }
            if (!changed) break; // early exit
        }
        // One more round: any further improvement means a negative cycle.
        for (int[] e : edges)
            if (dist[e[0]] != INF && dist[e[0]] + e[2] < dist[e[1]]) return null;
        int[] res = new int[n];
        for (int i = 0; i < n; i++) res[i] = dist[i] >= INF ? Integer.MAX_VALUE : (int) dist[i];
        return res;
    }
    // Same idea powers #232: limit rounds to K+1 and read from a copy of last round's array.
}
```

12. Recursion & Backtracking

#246 Subsets

MEDIUM

Time $O(n * 2^n)$ | Space $O(n)$

```
class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), res);
        return res;
    }
    // Every node of the recursion tree is a valid subset.
    private void backtrack(int[] nums, int start, List<Integer> cur, List<List<Integer>> res) {
        res.add(new ArrayList<>(cur)); // record a COPY
        for (int i = start; i < nums.length; i++) {
            cur.add(nums[i]); // choose
            backtrack(nums, i + 1, cur, res); // explore (only later elements -> no repeats)
            cur.remove(cur.size() - 1); // un-choose
        }
    }
}
```

#247 Subsets II

MEDIUM

Time $O(n * 2^n)$ | Space $O(n)$

```
class Solution {
    public List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums); // group duplicates together
        List<List<Integer>> res = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), res);
        return res;
    }
    private void backtrack(int[] nums, int start, List<Integer> cur, List<List<Integer>> res) {
        res.add(new ArrayList<>(cur));
        for (int i = start; i < nums.length; i++) {
            if (i > start && nums[i] == nums[i - 1]) continue; // skip duplicate at the same depth
            cur.add(nums[i]);
            backtrack(nums, i + 1, cur, res);
            cur.remove(cur.size() - 1);
        }
    }
}
```

#248 Permutations

MEDIUM

Time $O(n * n!)$ | Space $O(n)$

```
class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(nums, new boolean[nums.length], new ArrayList<>(), res);
        return res;
    }
    private void backtrack(int[] nums, boolean[] used, List<Integer> cur, List<List<Integer>> res) {
        if (cur.size() == nums.length) { res.add(new ArrayList<>(cur)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue; // each element used once per permutation
            used[i] = true; cur.add(nums[i]);
            backtrack(nums, used, cur, res);
            used[i] = false; cur.remove(cur.size() - 1);
        }
    }
}
```

#249 Permutations II

MEDIUM

Time $O(n * n!)$ | Space $O(n)$

```
class Solution {
    public List<List<Integer>> permuteUnique(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> res = new ArrayList<>();
        backtrack(nums, new boolean[nums.length], new ArrayList<>(), res);
        return res;
    }
    private void backtrack(int[] nums, boolean[] used, List<Integer> cur, List<List<Integer>> res) {
        if (cur.size() == nums.length) { res.add(new ArrayList<>(cur)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            // Duplicate rule: use equal numbers in a fixed order (previous copy must already be used).
            if (i > 0 && nums[i] == nums[i - 1] && !used[i - 1]) continue;
            used[i] = true; cur.add(nums[i]);
            backtrack(nums, used, cur, res);
            used[i] = false; cur.remove(cur.size() - 1);
        }
    }
}
```

#250 Combination Sum

MEDIUM

Time $O(2^t)$ | Space $O(t)$

```
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(candidates, target, 0, new ArrayList<>(), res);
        return res;
    }
    private void backtrack(int[] c, int rem, int start, List<Integer> cur, List<List<Integer>> res) {
        if (rem == 0) { res.add(new ArrayList<>(cur)); return; }
        for (int i = start; i < c.length; i++) {
            if (c[i] > rem) continue;
            cur.add(c[i]);
            backtrack(c, rem - c[i], i, cur, res); // pass i (not i+1): same number may repeat
            cur.remove(cur.size() - 1);
        }
    }
}
```

#251 Combination Sum II

MEDIUM

Time $O(2^n)$ | Space $O(n)$

```
class Solution {
    public List<List<Integer>> combinationSum2(int[] candidates, int target) {
        Arrays.sort(candidates);
        List<List<Integer>> res = new ArrayList<>();
        backtrack(candidates, target, 0, new ArrayList<>(), res);
        return res;
    }
    private void backtrack(int[] c, int rem, int start, List<Integer> cur, List<List<Integer>> res) {
        if (rem == 0) { res.add(new ArrayList<>(cur)); return; }
        for (int i = start; i < c.length; i++) {
            if (c[i] > rem) break; // sorted -> nothing further can fit
            if (i > start && c[i] == c[i - 1]) continue; // skip duplicate combos
            cur.add(c[i]);
            backtrack(c, rem - c[i], i + 1, cur, res); // each element used at most once
            cur.remove(cur.size() - 1);
        }
    }
}
```

#252 Combination Sum III

MEDIUM

Time $O(C(9,k))$ | Space $O(k)$

```
class Solution {
    public List<List<Integer>> combinationSum3(int k, int n) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(1, k, n, new ArrayList<>(), res);
        return res;
    }
    private void backtrack(int start, int k, int rem, List<Integer> cur, List<List<Integer>> res) {
        if (cur.size() == k) { if (rem == 0) res.add(new ArrayList<>(cur)); return; }
        for (int d = start; d <= 9 && d <= rem; d++) { // digits 1..9, each used once
            cur.add(d);
            backtrack(d + 1, k, rem - d, cur, res);
            cur.remove(cur.size() - 1);
        }
    }
}
```

#253 Letter Combinations of a Phone Number

MEDIUM

Time $O(4^n)$ | Space $O(n)$

```
class Solution {
    private static final String[] MAP = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv",
        "wxyz"};
    public List<String> letterCombinations(String digits) {
        List<String> res = new ArrayList<>();
        if (digits.isEmpty()) return res;
        backtrack(digits, 0, new StringBuilder(), res);
        return res;
    }
    private void backtrack(String d, int idx, StringBuilder sb, List<String> res) {
        if (idx == d.length()) { res.add(sb.toString()); return; }
        for (char c : MAP[d.charAt(idx) - '0'].toCharArray()) {
            sb.append(c);
            backtrack(d, idx + 1, sb, res);
            sb.deleteCharAt(sb.length() - 1);
        }
    }
}
```

#254 Generate Parentheses

MEDIUM

Time $O(4^n / \sqrt{n})$ | Space $O(n)$

```
class Solution {
    public List<String> generateParenthesis(int n) {
        List<String> res = new ArrayList<>();
        backtrack(n, 0, 0, new StringBuilder(), res);
        return res;
    }
    private void backtrack(int n, int open, int close, StringBuilder sb, List<String> res) {
        if (sb.length() == 2 * n) { res.add(sb.toString()); return; }
        if (open < n) { // may open while we have '(' left
            sb.append('('); backtrack(n, open + 1, close, sb, res); sb.deleteCharAt(sb.length() - 1);
        }
        if (close < open) { // may close only if there is an unmatched '('
            sb.append(')'); backtrack(n, open, close + 1, sb, res); sb.deleteCharAt(sb.length() - 1);
        }
    }
}
```

#255 Palindrome Partitioning

MEDIUM

Time $O(n * 2^n)$ | Space $O(n)$

```
class Solution {
    public List<List<String>> partition(String s) {
        List<List<String>> res = new ArrayList<>();
        backtrack(s, 0, new ArrayList<>(), res);
        return res;
    }
    private void backtrack(String s, int start, List<String> cur, List<List<String>> res) {
        if (start == s.length()) { res.add(new ArrayList<>(cur)); return; }
        for (int end = start; end < s.length(); end++) {
            if (isPal(s, start, end)) { // only cut where the piece is a palindrome
                cur.add(s.substring(start, end + 1));
                backtrack(s, end + 1, cur, res);
                cur.remove(cur.size() - 1);
            }
        }
    }
    private boolean isPal(String s, int l, int r) {
        while (l < r) if (s.charAt(l++) != s.charAt(r--)) return false;
        return true;
    }
}
```

#256 Word Search

MEDIUM

Same problem as #229 - **Word Search**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#257 N-Queens

HARD

Time $O(n!)$ | Space $O(n)$

```
class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> res = new ArrayList<>();
        int[] queens = new int[n]; // queens[row] = column
        backtrack(0, n, queens, new boolean[n], new boolean[2 * n], new boolean[2 * n], res);
        return res;
    }
    // cols[c], diag1[r-c+n], diag2[r+c] mark attacked lines in O(1).
    private void backtrack(int r, int n, int[] q, boolean[] cols, boolean[] d1, boolean[] d2, List<List<String>> res) {
        if (r == n) {
            List<String> board = new ArrayList<>();
            for (int i = 0; i < n; i++) {
                char[] row = new char[n];
                Arrays.fill(row, '.');
                row[q[i]] = 'Q';
                board.add(new String(row));
            }
            res.add(board);
            return;
        }
        for (int c = 0; c < n; c++) {
            if (cols[c] || d1[r - c + n] || d2[r + c]) continue;
            q[r] = c; cols[c] = d1[r - c + n] = d2[r + c] = true;
            backtrack(r + 1, n, q, cols, d1, d2, res);
            cols[c] = d1[r - c + n] = d2[r + c] = false; // backtrack
        }
    }
}
```

#258 N-Queens II

HARDTime $O(n!)$ | Space $O(n)$

```
class Solution {
    private int count = 0;
    public int totalNQueens(int n) {
        place(0, n, new boolean[n], new boolean[2 * n], new boolean[2 * n]);
        return count;
    }
    private void place(int r, int n, boolean[] cols, boolean[] d1, boolean[] d2) {
        if (r == n) { count++; return; } // full board = one solution
        for (int c = 0; c < n; c++) {
            if (cols[c] || d1[r - c + n] || d2[r + c]) continue;
            cols[c] = d1[r - c + n] = d2[r + c] = true;
            place(r + 1, n, cols, d1, d2);
            cols[c] = d1[r - c + n] = d2[r + c] = false;
        }
    }
}
```

#259 Sudoku Solver

HARDTime $O(9^{(\text{empty cells})})$ | Space $O(1)$

```
class Solution {
    public void solveSudoku(char[][] board) { solve(board); }
    private boolean solve(char[][] b) {
        for (int r = 0; r < 9; r++)
            for (int c = 0; c < 9; c++) {
                if (b[r][c] != '.') continue;
                for (char d = '1'; d <= '9'; d++) {
                    if (valid(b, r, c, d)) {
                        b[r][c] = d; // try
                        if (solve(b)) return true;
                        b[r][c] = '.'; // undo
                    }
                }
                return false; // no digit fits -> dead end
            }
        return true; // no empty cell left
    }
    private boolean valid(char[][] b, int r, int c, char d) {
        for (int i = 0; i < 9; i++) {
            if (b[r][i] == d || b[i][c] == d) return false;
            if (b[3 * (r / 3) + i / 3][3 * (c / 3) + i % 3] == d) return false; // 3x3 box
        }
        return true;
    }
}
```

#260 Rat in a Maze

MEDIUMTime $O(3^{(n^2)})$ | Space $O(n^2)$

```
class Solution {
    // maze[i][j] == 1 means open. Return all paths from (0,0) to (n-1,n-1) in lexicographic order.
    public ArrayList<String> findPath(int[][] maze) {
        ArrayList<String> res = new ArrayList<>();
        int n = maze.length;
        if (maze[0][0] == 0 || maze[n - 1][n - 1] == 0) return res;
        dfs(maze, 0, 0, new StringBuilder(), new boolean[n][n], res);
        return res;
    }
    private static final int[] DR = {1, 0, 0, -1};
    private static final int[] DC = {0, -1, 1, 0};
    private static final char[] DIR = {'D', 'L', 'R', 'U'}; // alphabetical -> results come out sorted
    private void dfs(int[][] m, int r, int c, StringBuilder path, boolean[][] vis, ArrayList<String> res) {
        {
            int n = m.length;
            if (r == n - 1 && c == n - 1) { res.add(path.toString()); return; }
            vis[r][c] = true;
            for (int k = 0; k < 4; k++) {
                int nr = r + DR[k], nc = c + DC[k];
                if (nr >= 0 && nc >= 0 && nr < n && nc < n && m[nr][nc] == 1 && !vis[nr][nc]) {
                    path.append(DIR[k]);
                    dfs(m, nr, nc, path, vis, res);
                    path.deleteCharAt(path.length() - 1);
                }
            }
        }
    }
}
```

```

        vis[r][c] = false; // backtrack: cell reusable on other paths
    }
}

```

#261 Restore IP Addresses

MEDIUM

Time O(1) (bounded) | Space O(1)

```

class Solution {
    public List<String> restoreIpAddresses(String s) {
        List<String> res = new ArrayList<>();
        backtrack(s, 0, new ArrayList<>(), res);
        return res;
    }
    private void backtrack(String s, int start, List<String> parts, List<String> res) {
        if (parts.size() == 4) {
            if (start == s.length()) res.add(String.join(".", parts)); // used all digits
            return;
        }
        for (int len = 1; len <= 3 && start + len <= s.length(); len++) {
            String seg = s.substring(start, start + len);
            if (seg.length() > 1 && seg.charAt(0) == '0') break; // no leading zeros
            if (Integer.parseInt(seg) > 255) break;
            parts.add(seg);
            backtrack(s, start + len, parts, res);
            parts.remove(parts.size() - 1);
        }
    }
}

```

#262 Letter Tile Possibilities

MEDIUM

Time O(n!) | Space O(n)

```

class Solution {
    public int numTilePossibilities(String tiles) {
        int[] cnt = new int[26];
        for (char c : tiles.toCharArray()) cnt[c - 'A']++;
        return dfs(cnt);
    }
    // Count every non-empty sequence: pick any available letter as the next character.
    private int dfs(int[] cnt) {
        int total = 0;
        for (int i = 0; i < 26; i++) {
            if (cnt[i] == 0) continue;
            cnt[i]--;
            total += 1 + dfs(cnt); // 1 for the sequence ending here + all its extensions
            cnt[i]++;
        }
        return total;
    }
}

```

#263 Matchsticks to Square

MEDIUM

Time O(4^n) | Space O(n)

```

class Solution {
    public boolean makesquare(int[] matchsticks) {
        int sum = 0;
        for (int m : matchsticks) sum += m;
        if (sum % 4 != 0) return false;
        Arrays.sort(matchsticks); // place big sticks first -> fail fast
        return dfs(matchsticks, matchsticks.length - 1, new int[4], sum / 4);
    }
    private boolean dfs(int[] m, int i, int[] sides, int target) {
        if (i < 0) return true; // all placed (sides must all equal target)
        for (int s = 0; s < 4; s++) {
            if (sides[s] + m[i] > target) continue;
            // Skip equivalent sides (same current length) to avoid repeated work.
            boolean dup = false;
            for (int t = 0; t < s; t++) if (sides[t] == sides[s]) { dup = true; break; }
            if (dup) continue;
            sides[s] += m[i];
            if (dfs(m, i - 1, sides, target)) return true;
            sides[s] -= m[i];
        }
        return false;
    }
}

```

```
}
```

#264 Partition to K Equal Sum Subsets

MEDIUMTime $O(k * 2^n)$ | Space $O(2^n)$

```
class Solution {
    public boolean canPartitionKSubsets(int[] nums, int k) {
        int sum = 0;
        for (int x : nums) sum += x;
        if (sum % k != 0) return false;
        int target = sum / k, n = nums.length;
        Arrays.sort(nums);
        if (nums[n - 1] > target) return false;
        // Bitmask DP: dp[mask] = sum of chosen numbers modulo target, or -1 if unreachable.
        int[] dp = new int[1 << n];
        Arrays.fill(dp, -1);
        dp[0] = 0;
        for (int mask = 0; mask < (1 << n); mask++) {
            if (dp[mask] == -1) continue;
            for (int i = 0; i < n; i++) {
                if ((mask & (1 << i)) != 0) continue; // already used
                if (dp[mask] + nums[i] > target) break; // sorted => no later number fits
                dp[mask | (1 << i)] = (dp[mask] + nums[i]) % target; // wrap to 0 when a bucket fills
            }
        }
        return dp[(1 << n) - 1] == 0;
    }
}
```

#265 Beautiful Arrangement

MEDIUMTime $O(k)$ (pruned) | Space $O(n)$

```
class Solution {
    private int count = 0;
    public int countArrangement(int n) {
        dfs(n, 1, new boolean[n + 1]);
        return count;
    }
    // Fill positions 1..n in order; number i fits position pos if i%pos==0 or pos%i==0.
    private void dfs(int n, int pos, boolean[] used) {
        if (pos > n) { count++; return; }
        for (int i = 1; i <= n; i++) {
            if (!used[i] && (i % pos == 0 || pos % i == 0)) {
                used[i] = true;
                dfs(n, pos + 1, used);
                used[i] = false;
            }
        }
    }
}
```

13. Greedy

#266 Best Time to Buy and Sell Stock II

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int maxProfit(int[] prices) {
        int profit = 0;
        // Collect every upward step: buying yesterday and selling today whenever price rises.
        for (int i = 1; i < prices.length; i++)
            if (prices[i] > prices[i - 1]) profit += prices[i] - prices[i - 1];
        return profit;
    }
}
```

#267 Jump Game

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public boolean canJump(int[] nums) {
        int farthest = 0;
        for (int i = 0; i < nums.length; i++) {
            if (i > farthest) return false; // stuck: can't even reach index i
            farthest = Math.max(farthest, i + nums[i]);
        }
        return true;
    }
}
```

#268 Jump Game II

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int jump(int[] nums) {
        // BFS by "levels": each jump extends the reachable window.
        int jumps = 0, curEnd = 0, farthest = 0;
        for (int i = 0; i < nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);
            if (i == curEnd) { jumps++; curEnd = farthest; } // must jump now
        }
        return jumps;
    }
}
```

#269 Gas Station

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int canCompleteCircuit(int[] gas, int[] cost) {
        int total = 0, tank = 0, start = 0;
        for (int i = 0; i < gas.length; i++) {
            int diff = gas[i] - cost[i];
            total += diff; tank += diff;
            if (tank < 0) { start = i + 1; tank = 0; } // can't pass i from 'start' -> restart after i
        }
        return total >= 0 ? start : -1; // enough overall fuel => start is valid
    }
}
```

#270 Assign Cookies

EASY

Time $O(n \log n)$ | Space $O(1)$

```
class Solution {
    public int findContentChildren(int[] g, int[] s) {
        Arrays.sort(g); Arrays.sort(s);
        int child = 0;
        for (int cookie = 0; cookie < s.length && child < g.length; cookie++)
            // smallest sufficient cookie goes to the least greedy child
            if (s[cookie] >= g[child]) child++;
        return child;
    }
}
```

#271 Lemonade Change

EASY

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public boolean lemonadeChange(int[] bills) {
        int five = 0, ten = 0;
        for (int b : bills) {
            if (b == 5) five++;
            else if (b == 10) { if (five == 0) return false; five--; ten++; }
            else { // b == 20: prefer giving 10+5 (keeps more 5s)
                if (ten > 0 && five > 0) { ten--; five--; }
                else if (five >= 3) five -= 3;
                else return false;
            }
        }
        return true;
    }
}
```

#272 Partition Labels

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public List<Integer> partitionLabels(String s) {
        int[] last = new int[26];
        for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;
        List<Integer> res = new ArrayList<>();
        int start = 0, end = 0;
        for (int i = 0; i < s.length(); i++) {
            // partition must reach the last use of every char in it
            end = Math.max(end, last[s.charAt(i) - 'a']);
            if (i == end) { res.add(end - start + 1); start = i + 1; }
        }
        return res;
    }
}
```

#273 Merge Triplets to Form Target Triplet

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public boolean mergeTriplets(int[][] triplets, int[] target) {
        boolean a = false, b = false, c = false;
        for (int[] t : triplets) {
            // Only triplets that never exceed the target in any position are safe to merge.
            if (t[0] > target[0] || t[1] > target[1] || t[2] > target[2]) continue;
            if (t[0] == target[0]) a = true;
            if (t[1] == target[1]) b = true;
            if (t[2] == target[2]) c = true;
        }
        return a && b && c;
    }
}
```

#274 Non-overlapping Intervals

MEDIUM

Same problem as #28 - *Non-overlapping Intervals*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#275 Minimum Number of Arrows to Burst Balloons

MEDIUM

Time $O(n \log n)$ | Space $O(1)$

```
class Solution {
    public int findMinArrowShots(int[][] points) {
        // Compare with Integer.compare (not subtraction) to avoid overflow.
        Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1])); // sort by end
        int arrows = 1, end = points[0][1];
        for (int i = 1; i < points.length; i++) {
            if (points[i][0] > end) { arrows++; end = points[i][1]; } // not covered by current arrow
        }
        return arrows;
    }
}
```

#276 Hand of Straights

MEDIUM

Time $O(n \log n)$ | Space $O(n)$

```
class Solution {
    public boolean isNStraightHand(int[] hand, int groupSize) {
        if (hand.length % groupSize != 0) return false;
        TreeMap<Integer, Integer> cnt = new TreeMap<>();
        for (int h : hand) cnt.merge(h, 1, Integer::sum);
        while (!cnt.isEmpty()) {
            int start = cnt.firstKey(); // smallest card must start a group
            for (int c = start; c < start + groupSize; c++) {
                Integer have = cnt.get(c);
                if (have == null) return false; // missing consecutive card
                if (have == 1) cnt.remove(c); else cnt.put(c, have - 1);
            }
        }
        return true;
    }
}
```

#277 Task Scheduler

MEDIUM

Same problem as #208 - *Task Scheduler*. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#278 Candy

HARD

Time $O(n)$ | Space $O(n)$

```
class Solution {
    public int candy(int[] ratings) {
        int n = ratings.length;
        int[] c = new int[n];
        Arrays.fill(c, 1);
        for (int i = 1; i < n; i++) // left neighbor rule
            if (ratings[i] > ratings[i - 1]) c[i] = c[i - 1] + 1;
        for (int i = n - 2; i >= 0; i--) // right neighbor rule (keep the larger)
            if (ratings[i] > ratings[i + 1]) c[i] = Math.max(c[i], c[i + 1] + 1);
        int total = 0;
        for (int x : c) total += x;
        return total;
    }
}
```

#279 Queue Reconstruction by Height

MEDIUM

Time $O(n^2)$ | Space $O(n)$

```
class Solution {
    public int[][] reconstructQueue(int[][] people) {
        // Tallest first (ties: smaller k first), then insert each person at index k.
        // Shorter people inserted later never affect the count seen by taller people.
        Arrays.sort(people, (a, b) -> a[0] != b[0] ? b[0] - a[0] : a[1] - b[1]);
        List<int[]> res = new ArrayList<>();
        for (int[] p : people) res.add(p[1], p);
        return res.toArray(new int[0][]);
    }
}
```

#280 Boats to Save People

MEDIUM

Same problem as #85 - **Boats to Save People**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

14. Dynamic Programming

#281 Climbing Stairs

EASY

Time O(n) | Space O(1)

```
class Solution {
    public int climbStairs(int n) {
        // ways(i) = ways(i-1) + ways(i-2) (last step was 1 or 2) -> Fibonacci.
        int a = 1, b = 1; // ways(0), ways(1)
        for (int i = 2; i <= n; i++) { int c = a + b; a = b; b = c; }
        return b;
    }
}
```

#282 Min Cost Climbing Stairs

EASY

Time O(n) | Space O(1)

```
class Solution {
    public int minCostClimbingStairs(int[] cost) {
        // dp[i] = min cost to reach step i. Top is index n (one past the last stair).
        int a = 0, b = 0; // dp[i-2], dp[i-1]
        for (int i = 2; i <= cost.length; i++) {
            int c = Math.min(b + cost[i - 1], a + cost[i - 2]);
            a = b; b = c;
        }
        return b;
    }
}
```

#283 House Robber

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int rob(int[] nums) {
        int take = 0, skip = 0; // best if last house robbed / not robbed
        for (int x : nums) {
            int newTake = skip + x; // rob this => previous must be skipped
            skip = Math.max(skip, take);
            take = newTake;
        }
        return Math.max(take, skip);
    }
}
```

#284 House Robber II

MEDIUM

Time O(n) | Space O(1)

```
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        if (n == 1) return nums[0];
        // Circle: first and last are adjacent -> solve twice, excluding one of them.
        return Math.max(line(nums, 0, n - 2), line(nums, 1, n - 1));
    }
    private int line(int[] a, int lo, int hi) {
        int take = 0, skip = 0;
        for (int i = lo; i <= hi; i++) {
            int nt = skip + a[i];
            skip = Math.max(skip, take);
            take = nt;
        }
        return Math.max(take, skip);
    }
}
```

#285 Longest Palindromic Substring (DP version)

MEDIUM

Time $O(n^2)$ | Space $O(n^2)$

```
class Solution {
    public String longestPalindrome(String s) {
        int n = s.length(), start = 0, best = 1;
        boolean[][] dp = new boolean[n][n]; // dp[i][j] = s[i..j] is a palindrome
        for (int j = 0; j < n; j++) {
            for (int i = 0; i <= j; i++) {
                // ends match AND inner part is a palindrome (or is tiny)
                if (s.charAt(i) == s.charAt(j) && (j - i < 2 || dp[i + 1][j - 1])) {
                    dp[i][j] = true;
                    if (j - i + 1 > best) { best = j - i + 1; start = i; }
                }
            }
        }
        return s.substring(start, start + best);
        // Note: #39 (expand around center) uses  $O(1)$  space and is the preferred interview answer.
    }
}
```

#286 Coin Change

MEDIUM

Time $O(\text{amount} * \text{coins})$ | Space $O(\text{amount})$

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1]; // dp[a] = fewest coins to make a
        Arrays.fill(dp, amount + 1); // "infinity"
        dp[0] = 0;
        for (int a = 1; a <= amount; a++)
            for (int c : coins)
                if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

#287 Coin Change II

MEDIUM

Time $O(\text{amount} * \text{coins})$ | Space $O(\text{amount})$

```
class Solution {
    public int change(int amount, int[] coins) {
        int[] dp = new int[amount + 1]; // dp[a] = number of combinations to make a
        dp[0] = 1;
        // Coins in the OUTER loop => each combination counted once (order ignored).
        for (int c : coins)
            for (int a = c; a <= amount; a++) dp[a] += dp[a - c];
        return dp[amount];
    }
}
```

#288 Partition Equal Subset Sum

MEDIUM

Time $O(n * \text{sum})$ | Space $O(\text{sum})$

```
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for (int x : nums) sum += x;
        if (sum % 2 != 0) return false;
        int target = sum / 2;
        boolean[] dp = new boolean[target + 1]; // dp[s] = some subset sums to s (0/1 knapsack)
        dp[0] = true;
        for (int x : nums)
            for (int s = target; s >= x; s--) // go DOWN so each number is used at most once
                dp[s] |= dp[s - x];
        return dp[target];
    }
}
```

#289 Target Sum

MEDIUM

Time $O(n * \text{sum})$ | Space $O(\text{sum})$

```
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        // P - N = target, P + N = total => P = (total + target) / 2. Count subsets with sum P.
        int total = 0;
        for (int x : nums) total += x;
        if (Math.abs(target) > total || (total + target) % 2 != 0) return 0;
        int p = (total + target) / 2;
        int[] dp = new int[p + 1];
        dp[0] = 1;
        for (int x : nums)
            for (int s = p; s >= x; s--) dp[s] += dp[s - x];
        return dp[p];
    }
}
```

#290 Decode Ways

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int numDecodings(String s) {
        int prev2 = 1, prev1 = s.charAt(0) == '0' ? 0 : 1; // ways for prefixes of length i-2, i-1
        for (int i = 2; i <= s.length(); i++) {
            int cur = 0;
            if (s.charAt(i - 1) != '0') cur += prev1; // single digit 1..9
            int two = Integer.parseInt(s.substring(i - 2, i));
            if (two >= 10 && two <= 26) cur += prev2; // two digits 10..26
            prev2 = prev1; prev1 = cur;
        }
        return prev1;
    }
}
```

#291 Longest Increasing Subsequence

MEDIUM

Time $O(n \log n)$ | Space $O(n)$

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        // tails[i] = smallest possible tail of an increasing subsequence of length i+1.
        int[] tails = new int[nums.length];
        int size = 0;
        for (int x : nums) {
            int lo = 0, hi = size;
            while (lo < hi) { // first tail >= x
                int mid = (lo + hi) / 2;
                if (tails[mid] < x) lo = mid + 1; else hi = mid;
            }
            tails[lo] = x; // replace or extend
            if (lo == size) size++;
        }
        return size;
    }
}
```

#292 Longest Common Subsequence

MEDIUM

Time $O(m*n)$ | Space $O(\min(m,n))$

```
class Solution {
    public int longestCommonSubsequence(String a, String b) {
        int[] prev = new int[b.length() + 1], cur = new int[b.length() + 1];
        for (int i = 1; i <= a.length(); i++) {
            for (int j = 1; j <= b.length(); j++) {
                // match: extend diagonal
                if (a.charAt(i - 1) == b.charAt(j - 1)) cur[j] = prev[j - 1] + 1;
                else cur[j] = Math.max(prev[j], cur[j - 1]); // skip one char
            }
            int[] t = prev; prev = cur; cur = t; // rolling rows
        }
        return prev[b.length()];
    }
}
```

#293 Edit Distance

MEDIUM

Time $O(m*n)$ | Space $O(m*n)$

```
class Solution {
    public int minDistance(String a, String b) {
        int m = a.length(), n = b.length();
        int[][] dp = new int[m + 1][n + 1]; // dp[i][j] = edits to turn a[0..i) into b[0..j)
        for (int i = 0; i <= m; i++) dp[i][0] = i; // delete all
        for (int j = 0; j <= n; j++) dp[0][j] = j; // insert all
        for (int i = 1; i <= m; i++)
            for (int j = 1; j <= n; j++) {
                if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1];
                else dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], // replace
                                              Math.min(dp[i - 1][j], dp[i][j - 1])); // delete / insert
            }
        return dp[m][n];
    }
}
```

#294 Unique Paths

MEDIUM

Time $O(m*n)$ | Space $O(n)$

```
class Solution {
    public int uniquePaths(int m, int n) {
        int[] dp = new int[n];
        Arrays.fill(dp, 1); // first row: one way
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++) dp[j] += dp[j - 1]; // from above + from left
        return dp[n - 1];
    }
}
```

#295 Unique Paths II

MEDIUM

Time $O(m*n)$ | Space $O(n)$

```
class Solution {
    public int uniquePathsWithObstacles(int[][] g) {
        int n = g[0].length;
        int[] dp = new int[n];
        dp[0] = g[0][0] == 1 ? 0 : 1;
        for (int[] row : g)
            for (int j = 0; j < n; j++) {
                if (row[j] == 1) dp[j] = 0; // obstacle: no path through here
                else if (j > 0) dp[j] += dp[j - 1];
            }
        return dp[n - 1];
    }
}
```

#296 Minimum Path Sum

MEDIUM

Time $O(m \cdot n)$ | Space $O(n)$

```
class Solution {
    public int minPathSum(int[][] g) {
        int m = g.length, n = g[0].length;
        int[] dp = new int[n];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) {
                if (i == 0 && j == 0) dp[j] = g[0][0];
                else if (i == 0) dp[j] = dp[j - 1] + g[i][j];
                else if (j == 0) dp[j] = dp[j] + g[i][j];
                else dp[j] = Math.min(dp[j], dp[j - 1]) + g[i][j]; // from above vs from left
            }
        return dp[n - 1];
    }
}
```

#297 Triangle

MEDIUM

Time $O(n^2)$ | Space $O(n)$

```
class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {
        int n = triangle.size();
        int[] dp = new int[n + 1]; // bottom-up: start from the last row
        for (int i = n - 1; i >= 0; i--)
            for (int j = 0; j <= i; j++)
                dp[j] = triangle.get(i).get(j) + Math.min(dp[j], dp[j + 1]);
        return dp[0];
    }
}
```

#298 Word Break

MEDIUM

Time $O(n^2)$ | Space $O(n)$

```
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        Set<String> dict = new HashSet<>(wordDict);
        boolean[] dp = new boolean[s.length() + 1]; // dp[i] = s[0..i] can be segmented
        dp[0] = true;
        for (int i = 1; i <= s.length(); i++)
            for (int j = 0; j < i; j++)
                if (dp[j] && dict.contains(s.substring(j, i))) { dp[i] = true; break; }
        return dp[s.length()];
    }
}
```

#299 Word Break II

HARD

Time $O(2^n)$ worst | Space $O(2^n)$

```
class Solution {
    private Map<Integer, List<String>> memo = new HashMap<>();
    public List<String> wordBreak(String s, List<String> wordDict) {
        return dfs(s, 0, new HashSet<>(wordDict));
    }
    // All sentences that can be built from s[start..]. Memoized by start index.
    private List<String> dfs(String s, int start, Set<String> dict) {
        if (memo.containsKey(start)) return memo.get(start);
        List<String> res = new ArrayList<>();
        if (start == s.length()) { res.add(""); return res; }
        for (int end = start + 1; end <= s.length(); end++) {
            String word = s.substring(start, end);
            if (!dict.contains(word)) continue;
            for (String rest : dfs(s, end, dict))
                res.add(rest.isEmpty() ? word : word + " " + rest);
        }
        memo.put(start, res);
        return res;
    }
}
```

#300 Combination Sum IV

MEDIUM

Time $O(\text{target} * n)$ | Space $O(\text{target})$

```
class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] dp = new int[target + 1]; // dp[t] = number of ORDERED sequences summing to t
        dp[0] = 1;
        // Target in the OUTER loop => different orders count separately (contrast with Coin Change II).
        for (int t = 1; t <= target; t++)
            for (int x : nums)
                // overflow is guaranteed not to matter per the problem
                if (x <= t) dp[t] += dp[t - x];
        return dp[target];
    }
}
```

#301 House Robber III

MEDIUM

Same problem as **#193 - House Robber III**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#302 Maximum Product Subarray

MEDIUM

Same problem as **#6 - Maximum Product Subarray**. Solution above; re-solve it from memory without looking, this repeat is for pattern recognition.

#303 Best Time to Buy and Sell Stock with Cooldown

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int maxProfit(int[] prices) {
        // State machine: hold (own a stock), sold (just sold today), rest (no stock, free to buy).
        int hold = Integer.MIN_VALUE, sold = 0, rest = 0;
        for (int p : prices) {
            int prevSold = sold;
            sold = hold + p; // sell today
            hold = Math.max(hold, rest - p); // keep holding, or buy today (from rest)
            rest = Math.max(rest, prevSold); // stay idle, or cooldown ends
        }
        return Math.max(sold, rest);
    }
}
```

#304 Best Time to Buy and Sell Stock with Transaction Fee

MEDIUM

Time $O(n)$ | Space $O(1)$

```
class Solution {
    public int maxProfit(int[] prices, int fee) {
        int cash = 0, hold = -prices[0]; // best profit with no stock / holding a stock
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i] - fee); // sell (pay fee once, at selling)
            hold = Math.max(hold, cash - prices[i]); // buy
        }
        return cash;
    }
}
```

#305 Interleaving String

MEDIUM

Time $O(m \cdot n)$ | Space $O(n)$

```
class Solution {
    public boolean isInterleave(String s1, String s2, String s3) {
        int m = s1.length(), n = s2.length();
        if (m + n != s3.length()) return false;
        // dp[j] for current i: s3[0..i+j] built from s1[0..i), s2[0..j)
        boolean[] dp = new boolean[n + 1];
        for (int i = 0; i <= m; i++)
            for (int j = 0; j <= n; j++) {
                if (i == 0 && j == 0) dp[j] = true;
                else {
                    boolean fromS1 = i > 0 && dp[j] && s1.charAt(i - 1) == s3.charAt(i + j - 1);
                    boolean fromS2 = j > 0 && dp[j - 1] && s2.charAt(j - 1) == s3.charAt(i + j - 1);
                    dp[j] = fromS1 || fromS2;
                }
            }
        return dp[n];
    }
}
```

#306 Distinct Subsequences

HARD

Time $O(m \cdot n)$ | Space $O(n)$

```
class Solution {
    public int numDistinct(String s, String t) {
        // dp[j] = ways s-prefix contains t[0..j) as a subsequence
        int[] dp = new int[t.length() + 1];
        dp[0] = 1; // empty t: exactly one way
        for (int i = 0; i < s.length(); i++)
            for (int j = t.length(); j >= 1; j--) // go DOWN so dp[j-1] is still the previous row
                // use this s char to match t[j-1]
                if (s.charAt(i) == t.charAt(j - 1)) dp[j] += dp[j - 1];
        return dp[t.length()];
    }
}
```

#307 Longest Palindromic Subsequence

MEDIUM

Time $O(n^2)$ | Space $O(n^2)$

```
class Solution {
    public int longestPalindromeSubseq(String s) {
        int n = s.length();
        int[][] dp = new int[n][n]; // dp[i][j] = LPS length inside s[i..j]
        for (int i = n - 1; i >= 0; i--) {
            dp[i][i] = 1;
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) dp[i][j] = dp[i + 1][j - 1] + 2; // wrap both ends
                else dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]); // drop one end
            }
        }
        return dp[0][n - 1];
    }
}
```

#308 Regular Expression Matching

HARDTime $O(m*n)$ | Space $O(m*n)$

```
class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m + 1][n + 1]; // dp[i][j] = s[0..i) matches p[0..j)
        dp[0][0] = true;
        for (int j = 2; j <= n; j++) // patterns like a*b*c* can match empty string
            if (p.charAt(j - 1) == '*') dp[0][j] = dp[0][j - 2];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                char pc = p.charAt(j - 1);
                if (pc == '*') {
                    dp[i][j] = dp[i][j - 2]; // zero occurrences
                    char prev = p.charAt(j - 2);
                    // one more occurrence
                    if (prev == '.' || prev == s.charAt(i - 1)) dp[i][j] |= dp[i - 1][j];
                } else if (pc == '.' || pc == s.charAt(i - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                }
            }
        }
        return dp[m][n];
    }
}
```

#309 Burst Balloons

HARDTime $O(n^3)$ | Space $O(n^2)$

```
class Solution {
    public int maxCoins(int[] nums) {
        int n = nums.length;
        int[] a = new int[n + 2]; // pad with 1s on both ends
        a[0] = a[n + 1] = 1;
        for (int i = 0; i < n; i++) a[i + 1] = nums[i];
        // dp[l][r] = best coins bursting everything strictly between l and r
        int[][] dp = new int[n + 2][n + 2];
        for (int len = 2; len <= n + 1; len++)
            for (int l = 0; l + len <= n + 1; l++) {
                int r = l + len;
                for (int k = l + 1; k < r; k++) // k = the LAST balloon burst in (l, r)
                    dp[l][r] = Math.max(dp[l][r], dp[l][k] + a[l] * a[k] * a[r] + dp[k][r]);
            }
        return dp[0][n + 1];
    }
}
```

#310 Maximum Profit in Job Scheduling

HARDTime $O(n \log n)$ | Space $O(n)$

```
class Solution {
    public int jobScheduling(int[] startTime, int[] endTime, int[] profit) {
        int n = startTime.length;
        int[][] jobs = new int[n][3];
        for (int i = 0; i < n; i++) jobs[i] = new int[]{startTime[i], endTime[i], profit[i]};
        Arrays.sort(jobs, (x, y) -> x[1] - y[1]); // sort by end time
        int[] dp = new int[n + 1]; // dp[i] = best profit using first i jobs
        for (int i = 1; i <= n; i++) {
            int[] job = jobs[i - 1];
            // Binary search: last job (among first i-1) whose end <= this job's start.
            int lo = 0, hi = i - 1;
            while (lo < hi) {
                int mid = (lo + hi + 1) / 2;
                if (jobs[mid - 1][1] <= job[0]) lo = mid; else hi = mid - 1;
            }
            dp[i] = Math.max(dp[i - 1], dp[lo] + job[2]); // skip job vs take job
        }
        return dp[n];
    }
}
```